



常州工学院
CHANGZHOU INSTITUTE OF TECHNOLOGY

课程大作业说明书

大数据与云计算（Q）

基于 Hadoop-Spark 的电商行为智能分析系统

二级学院（直属学部）：计算机信息工程学院

专 业：软件工程 班级：23 软件一

学生姓名：许子祺 学号：23030327

学生姓名：莫文涛 学号：23030315

学生姓名：钱季清 学号：23030326

学生姓名：王奕泽 学号：23250123

2026 年 6 月

基于 Hadoop-Spark 的电商行为智能分析系统

摘要

本文围绕电商用户行为数据分析课题，构建了一套基于 Docker ARM64、Hadoop HDFS/YARN、Spark SQL、Flask API 与 React 前端的本机大数据分析实验系统。系统以 Kaggle 电商行为数据为主要输入，完成原始 CSV 数据采样、HDFS 上传、Spark 分布式读取、字段标准化、异常值过滤、重复事件去重、会话事实表生成、指标缓存发布和前端可视化展示。业务层面实现了基础看板、转化漏斗、用户行为路径（含事件转移矩阵）、特征集市、需求预测（双尺度 AR-MLP）、商品关联图谱（Support/Confidence/Lift）、Cohort 留存、商品组合分析（HHI 浓缩度）、购物车挽回、收入归因、运营优化（MILP 与贪心降级）、个性化推荐（含 LR/GBT 排序器与 precision/recall/NDCG 评估指标）、异常检测（Robust Z-Score）、生命周期分层、受控自然语言查询、实验分流（含 SRM 卡方检验与 AUUC 评估）、决策引擎（决策自动生成）和 Ops 证据面板等模块。各模块统一复用清洗后的事件表、会话事实表和日粒度特征表，保证不同页面中的行数、收入、转化率、留存率、推荐置信度和异常分级口径一致。

在工程实现上，本文重点解决了本机 ARM Mac 环境下 Hadoop/Spark 集群复现、端口冲突、跨架构镜像不可用、Spark History 日志过大、driver 端全量 collect 造成内存风险等问题。系统采用 YARN client mode 作为第一阶段提交方式，保留 Flask 读取本地缓存的便利性；同时使用 AQE、合理 shuffle 分区、MEMORY_AND_DISK 缓存、Parquet 清洗层、preview JSON 上界和模块级质量门禁降低运行风险。实验部分对比 local Spark、YARN-only、YARN+AQE、YARN+ 算法限流和 YARN+Parquet 多组配置，并补充 1%/5% 用户级样本与典型模块 benchmark。结果表明，仅把任务提交到 YARN 并不能自动提升性能或避免 OOM；真正有效的优化来自 Spark SQL 执行计划、列式数据布局、算法输出边界和可追溯的运行证据。最终系统能够在课程设计环境中完成从分布式存储、Spark 计算、后端接口到前端可视化和报告分析的完整闭环。

关键词：Hadoop YARN；Spark SQL；Docker ARM64；电商行为分析；内存优化；Benchmark

目 录

第 1 章 绪论（课题背景、现状）	1
1.1 课题背景	1
1.2 国内外现状	1
1.3 课题问题定义	2
1.4 本文结构	2
第 2 章 相关技术	3
2.1 大数据基础设施技术	3
2.1.1 Docker 容器化与 ARM64 集群编排	3
2.1.2 HDFS 分布式存储与 Parquet 列式格式	3
2.1.3 YARN 资源管理与 Spark 部署模式	5
2.2 分布式计算引擎与查询优化	5
2.2.1 Spark SQL 与 Catalyst 优化器	5
2.2.2 AQE 自适应查询执行	6
2.3 服务层与展示层技术选型	7
2.3.1 Flask 异步架构与 SQLite 持久化	7
2.3.2 React 前端与 Ops 证据面板	8
第 3 章 需求分析与实验对照设计	9
3.1 系统总体目标与工程验收指标	9
3.2 系统功能与非功能性需求分析	10
3.3 1% 与 5% 多配置梯度实验对照设计	10
3.4 数据清理与存储安全需求	11
第 4 章 系统架构与数据工程实现	13
4.1 总体架构与环境配置	13
4.2 分布式提交链路与内存优化控制	14
4.2.1 YARN Client 提交流程	14
4.2.2 Driver 端内存保护	15
4.3 数据清洗与质量门禁	16
4.3.1 字段标准化与类型转换	16
4.4 会话与特征工程	17
4.4.1 会话事实表与转化漏斗	17

4.4.2	高频行为路径与转移矩阵.....	18
4.4.3	双粒度特征集市.....	18
第 5 章	业务分析与智能决策模块实现	20
5.1	深度业务预测与需求估计	20
5.1.1	双尺度需求预测与 WAPE 精度度量	20
5.1.2	商品关联图谱挖掘与置信度评价.....	21
5.2	用户生命周期与留存分析	22
5.2.1	Cohort 留存分析与购物车挽回.....	22
5.2.2	RFM 生命周期分群模型.....	23
5.3	归因、优化与推荐决策	25
5.3.1	收入归因与运营优化.....	25
5.3.2	多路召回与轻量排序推荐系统.....	28
5.4	商品组合智能与用户旅程分析	31
5.4.1	HHI 集中度与品类结构健康度.....	31
5.4.2	用户旅程与转移矩阵.....	33
5.5	增长实验平台与因果推断框架	35
5.5.1	确定性哈希分桶与样本比例检验	35
5.6	自动决策引擎与受控查询	37
5.6.1	异常驱动的自动干预决策.....	38
第 6 章	运维风险控制与 Benchmark 评估.....	40
6.1	异常检测与实时预警	40
6.1.1	Robust Z-Score 异常检测	40
6.2	生命周期分层与实验分流	41
6.2.1	生命周期分层与运维干预	41
6.2.2	实验分流的运维稳定机制与护栏监控.....	41
6.3	可追溯 Ops 证据链.....	42
6.3.1	Ops 证据看板与运行元数据	42
6.4	多维度 Benchmark 实验结果与对比分析.....	42
6.4.1	多配置对照实验结果.....	42
第 7 章	致谢	46
参考文献		47

第 1 章 绪论（课题背景、现状）

1.1 课题背景

电子商务平台每天会产生大量用户行为日志，包括浏览、加购、移除购物车、购买、商品、类目、品牌、价格、用户和会话等字段。单机脚本可以完成小样本统计，但当数据量从几十 KB 的演示文件增长到数 GB 的真实日志时，原有处理方式会暴露出三个问题：第一，CSV 扫描成本高，重复读取会拖慢整个分析流程；第二，复杂分析模块容易把明细结果收集到 driver 端，造成 Python list 或 JSON payload 过大；第三，关联分析、推荐候选、异常检测等模块存在组合膨胀风险，数据规模增加后可能出现长尾 task、shuffle 放大和内存溢出。

本课题的目标不是简单把程序放进 Docker，也不是仅把 Spark master 改成 YARN，而是围绕“真实大数据实验是否可复现、可解释、可对比”设计一套本机稳妥型实验系统。系统使用 Kaggle 电商行为数据作为原始输入，保留 Oct 和 Nov 两个月 CSV 原始数据，通过用户级采样构造 1% 与 5% 实验集，再在 Docker ARM64 Hadoop YARN 集群上运行 PySpark pipeline。最终产出包括 HDFS 数据快照、Spark 运行记录、质量报告、前端 Ops 页和 benchmark 对比结果。

1.2 国内外现状

大数据批处理的经典起点是 MapReduce。MapReduce 将海量数据处理抽象为 Map 和 Reduce 两个阶段，通过分布式文件系统、任务重试和数据本地性降低大规模批处理的工程复杂度^[1]。Hadoop 生态在此基础上形成了 HDFS、MapReduce 和 YARN 等组件。YARN 将资源管理从具体计算框架中抽离出来，由 ResourceManager、NodeManager 和 ApplicationMaster 协同完成资源调度，使同一集群能够承载不同计算引擎^[2]。

Spark 在 RDD 模型中引入内存计算、lineage 容错和数据复用，解决了 MapReduce 在迭代计算、交互式分析和多阶段作业中的效率问题^[3-4]。Spark SQL 又通过 Catalyst 优化器、DataFrame API、列式执行和谓词下推，使结构化数据分析更加接近数据库优化器的工作方式^[5]。在现代 Spark 应用中，是否启用 AQE、是否使用 Parquet、是否合理控制 shuffle 分区和 driver collect，往往比“是否使用集群”本身更影响稳定性。

同时，学术界和工业界已经认识到大数据系统性能不能只依赖静态经验参数。

Starfish 提出 profile-driven tuning, 强调先收集运行画像, 再根据证据调优^[6]; Skew-Tune 关注数据倾斜和长尾 task 对整体作业时间的影响^[7]; Themis 则提醒系统设计不能盲目追求全内存, 应当接受合理 spill 和磁盘中间结果^[8]。这些思想与本课题的实验结论一致: 仅增加 YARN 不能自动避免 OOM, 必须把资源调度、执行计划、数据布局和算法输出边界一起纳入设计。

1.3 课题问题定义

本项目早期使用 local Spark 和小样本 JSON 缓存, 适合演示页面功能, 但不足以支撑“大数据与云计算”课程中的集群实验和论文结论。前端出现红色长条警告、滚动异常等问题时, 根因并不只在页面布局, 也与后端一次性输出过多明细、质量状态不清晰和运行证据分散有关。因此, 本课题将问题拆成四类:

1. **数据规模问题**: 从 87.4KB smoke 文件扩展到真实 Oct/Nov 原始 CSV, 并构造 1% 和 5% 用户级样本。
2. **运行环境问题**: 从 local[*] 升级为 Docker Hadoop YARN 实验集群, 同时兼容 ARM64 Mac, 避免下载 amd64 Hadoop 镜像。
3. **算法内存问题**: 限制 driver 端 full collect, 将明细输出收敛为 Parquet 与 preview JSON, 并对关联、推荐、异常等模块加入上界。
4. **证据展示问题**: 把 benchmark、HDFS 输入、YARN app id、质量状态和清理策略汇总到前端 Ops 页与 LaTeX 报告中, 减少答辩时翻终端的成本。

1.4 本文结构

本文第 2 章介绍相关技术; 第 3 章给出需求分析与实验对照设计; 第 4 章详述系统架构与数据工程实现; 第 5 章详述业务分析与智能决策模块实现; 第 6 章展示运维风险控制与 Benchmark 评估; 第 7 章为致谢; 最后列出参考文献。

第 2 章 相关技术

2.1 大数据基础设施技术

2.1.1 Docker 容器化与 ARM64 集群编排

本项目的实验环境使用 Docker Compose 组织一套可重复启动、可清理、可展示的本机实验集群^[9]。采用容器化方案的原因有三。第一，Hadoop、Spark、Java、Python、Flask 和前端 Node 环境依赖较多，全部安装在宿主机上容易出现版本冲突和路径污染，影响实验复现。第二，课程答辩需要展示 HDFS、YARN、Spark History 等服务的运行状态，容器化可以将这些服务固定在明确的网络拓扑上。第三，本机为 ARM 架构 Mac，现成的 Hadoop 镜像大多仅支持 amd64 架构，直接使用会触发跨架构模拟，启动缓慢且不稳定。

因此，项目采用自建 ARM64 Hadoop 镜像，而非常见的 amd64 镜像。镜像构建时只保留 Hadoop 运行所需的 Java、SSH、rsync、bash、procpss、curl 以及精简后的 /opt/hadoop，避免将旧容器中的 HBase、Hive、日志和 HDFS 数据目录等一并打包。最终方案采用 Dockerfile 构建，使镜像来源可追溯，也更符合课程实验中可复现性的要求。

Docker Compose 使用 yarn-lab profile 组织 Hadoop 与 Spark 服务。该 profile 下包含 NameNode、DataNode、SecondaryNameNode、ResourceManager、NodeManager、Spark Client 和 Spark History Server。它们在同一个 Compose 网络中通过服务名互相访问，例如 Spark 作业通过 `hdfs://` 协议访问 NameNode，通过 YARN ResourceManager 申请 Executor 容器。项目同时保留 local-lab profile，用于不启动完整 YARN 时进行 Flask 与 React 的前端开发调试。

在本项目中，Docker 的作用不仅是“打包运行环境”，更重要的是提供一个可控的课程实验边界：哪些服务启动、数据写到哪里、日志如何清理，都能在 Compose 配置中明确说明。

2.1.2 HDFS 分布式存储与 Parquet 列式格式

HDFS 是 Hadoop 生态的分布式文件系统，适合存储大规模结构化与半结构化数据。在本项目中，HDFS 主要承担三类数据的存储。第一类是输入数据，即从真实 Kaggle 原始 CSV 中抽样得到的用户级样本。样本上传到 HDFS 的 /user/course/ 目录下，Spark 作业通过配置文件读取这些路径。第二类是清洗后的中间结果，例

表 2.1 Docker YARN 实验栈中的容器分工

容器或服务	所属层	作用
yarn-namenode	HDFS	保存 HDFS 目录树、文件元数据和 block 位置信息，对外提供 NameNode UI 与 RPC。
yarn-datanode-1/2	HDFS	保存实际数据块，模拟分布式文件系统中的多个存储节点。
secondarynamenode	HDFS	定期辅助 NameNode 做 checkpoint，降低元数据恢复压力。
resourcemanager	YARN	管理集群资源、队列和 application 生命周期。
nodemanager-1/2	YARN	在节点侧启动 Executor container，向 ResourceM-anager 汇报资源与运行状态。
spark-client	Spark	作为提交入口运行 spark-submit，在 Client Mode 下也承载 Spark Driver。
spark-history-server	Spark	从 hdfs:///spark-history 读取 event log，展示 stage、task、shuffle 和 executor 证据。
web-yarn	服务层	启动 Flask 后端，读取缓存与 SQLite 作业记录，对外提供 RESTful 接口。

如 Parquet 分区目录。相比反复扫描 CSV，Parquet 可以保存列类型、压缩信息和分区结构，更适合作为后续实验输入。第三类是 Spark History event log，保存在 /spark-history，供 Spark History Server 回放作业执行过程。

从物理存储角度看，CSV 格式是典型的行式文本。当需要进行多维过滤（例如仅筛选特定商品品牌）或执行统计指标（例如计算总 GMV）时，Spark 必须读取整行文本并对所有无用列执行序列化与反序列化，极大浪费了 I/O 吞吐和计算资源。与之对应，Parquet 列式存储提供了强类型 Schema 固化和高效的压缩机制。通过列裁剪（Column Projection）与谓词下推（Predicate Pushdown），Spark 在读取 Parquet 文件时可以直接跳过无关列的物理读取，并利用文件头中的元数据过滤掉不符合条件的行组（Row Groups），从而成倍降低网络传输与内存压力。

在代码层面，Spark 对文件存储格式的抽象非常彻底。无论是本地路径还是 hdfs:// 路径，Spark 均通过统一的 DataFrame Reader API 进行读取。项目只需在配置文件中修改输入路径，即可实现从本地到分布式存储的无缝切换，大幅降低了工程迁移与验证成本。

2.1.3 YARN 资源管理与 Spark 部署模式

YARN (Yet Another Resource Negotiator) 是 Hadoop 的分布式资源管理器, 在 Spark 体系结构中, YARN 凭借对集群资源的弹性调度、应用队列隔离以及与 Hadoop 存储层的天然契合, 成为最主流的大数据生产环境部署方案。

2.1.3.1 ResourceManager 与 NodeManager 协同

YARN 通过 ResourceManager 统一协调集群的计算资源。ResourceManager 维护全局资源视图, 负责接收应用提交请求并将资源以 Container 形式分配给各个应用。NodeManager 运行在每个工作节点上, 负责启动和管理本节点的 Container, 定期向 ResourceManager 汇报资源使用与健康状态。二者协同构成了 YARN 的主从式资源管理架构: ResourceManager 作为主控节点做出全局调度决策, NodeManager 作为执行节点承载实际计算任务。

2.1.3.2 Client Mode 与 Cluster Mode 对比

本项目采用 YARN Client Mode 作为实验模式。在 Client Mode 下, 控制整个作业生命周期的 Spark Driver 进程运行在提交任务的 Spark Client 容器中, 而实际负责数据读取与运算的 Spark Executor 则分配在 YARN 各个 NodeManager 的 Container 内。这种设计在方便开发调试的同时, 保证了后端服务能够直接检索 Driver 写入的指标缓存文件。

若切换至 YARN Cluster Mode, Driver 进程将直接运行在 YARN 的 Application-Master 内部, 解除 Driver 对本地物理节点的绑定, 实现完全的分布式容错。两种模式的选择取决于应用场景: Client Mode 适合交互式调试与课程实验, Cluster Mode 适合生产环境的长时间批处理作业。

2.2 分布式计算引擎与查询优化

2.2.1 Spark SQL 与 Catalyst 优化器

Spark SQL 的核心计算效率得益于 Catalyst 优化器。Catalyst 基于 Scala 的函数式编程特性, 为结构化查询语言提供了一个可扩展的执行计划优化框架。

2.2.1.1 四阶段优化流程

Catalyst 的工作流可以划分为四个阶段:

1. 分析阶段 (Analysis): 将抽象语法树 (AST) 转换为包含字段类型与表结构

的未解析逻辑计划（Unresolved Logical Plan），并通过 Catalog 进行元数据绑定。

2. **逻辑计划优化（Logical Optimization）**：应用一系列基于规则的优化方法（RBO, Rule-Based Optimization），包括常量折叠（Constant Folding）、算子下推（Operator Pushdown）、投影剪裁（Projection Pruning）和表达式简化。
3. **物理计划生成（Physical Planning）**：在基于成本的优化（CBO, Cost-Based Optimization）指导下，生成多个可行的物理计划，并根据数据分布与算子开销模型选出最优的物理执行计划。
4. **代码生成（Code Generation）**：利用 Java 字节码技术（Janino 编译器），将选中的物理算子动态编译成紧凑的 Java 代码，从而减少虚函数调用开销，提高 CPU 执行效率。

2.2.1.2 谓词下推与列裁剪

谓词下推是 Catalyst 优化器中最核心的规则之一。当用户编写如下 Spark 语句：

```
df.filter(F.col("brand") == "apple").select("user_id", "price")
```

Catalyst 不会在读取全部数据后再过滤品牌，而是将 Filter 算子直接”下推”到数据源端。在使用 Parquet 等列式存储文件时，该规则允许 Spark 底层读取器在读取阶段只加载 brand、user_id 和 price 三列，并根据行组的元数据统计信息直接跳过非 apple 品牌的行组，使得网络传输数据量大幅降低。

2.2.2 AQE 自适应查询执行

在大数据计算中，数据倾斜（Data Skew）是影响系统并行效能的主要瓶颈。电商行为数据具有典型的幂律分布特征，即少数热门品牌和头部类目的事件总数远超长尾类别。当执行分布式连接（Join）或按品牌分组聚合（GroupBy）时，相同的 Key 必须通过 Shuffle 汇总到同一个 Executor 节点上进行处理，导致该节点内存占用飙升、CPU 满载，形成典型的”长尾 Task”，严重拖慢整体计算流程。

Spark SQL 引入的自适应查询执行（AQE, Adaptive Query Execution）改变了传统的静态查询计划优化方式。AQE 在每个 Shuffle Map 阶段结束时插入”栅栏（Barrier）”，实时收集物理分区的数据大小、记录行数等统计特征，动态地在运行时优化后续执行计划：

2.2.2.1 小分区合并

根据 Shuffle 写入的实际数据量, AQE 自动将细碎的、小数据量分区合并为更合理的大分区, 避免产生大量小 Task 导致的任务调度开销。

2.2.2.2 Join 转换优化

如果运行时发现原本需要执行昂贵 Shuffle 的一侧 DataFrame 经过清洗过滤后实际体积小于广播阈值, AQE 会自动将 SortMergeJoin 转换为轻量级的 BroadcastHashJoin, 减少不必要的 Shuffle 操作。

2.2.2.3 数据倾斜处理

当检测到某个 Shuffle 分区的数据量远大于其他分区均值时, AQE 会自动将倾斜分区拆分为若干子分区, 同时将 Join 另一侧的对应数据进行复制关联, 实现倾斜数据的分布式并行处理, 有效规避 Executor 端的内存溢出。

2.3 服务层与展示层技术选型

2.3.1 Flask 异步架构与 SQLite 持久化

Flask 在项目中承担服务层角色, 负责将 Spark 离线计算产出的指标、作业日志和运行证据封装成标准 RESTful 接口。每个接口返回统一的 ApiEnvelope 结构, 包含状态码、提示语和数据体。为保证 Web 后端的吞吐和可用性, 计算层与接口响应采用异步解耦的设计方案。

2.3.1.1 JobService 异步解耦

当用户通过前端发出数据刷新请求时, Flask 并不在主进程中同步执行 Spark 管道, 而是将计算作业封装为一个后台 Job, 通过 JobService 将作业的元数据(包括任务标识、创建时间、当前状态等)持久化至 SQLite 本地数据库中。后台单独运行的 SparkPipelineRunner 进程异步被唤醒执行。计算完成后, 更新 SQLite 中的作业状态, 并将质量门禁指标、Spark 运行证据以及计算结果路径写入 SQLite 供后续检索。

2.3.1.2 MetricCache 两阶段退化检索

在数据读取上, Flask 后端引入了 MetricCache (指标多维缓存) 以提供毫秒级的客户端响应。为了处理各种极端情况, MetricCache 设计了两阶段退化检索机制:

1. **第一阶段：多维立方体优先检索**——离线计算管道运行时，会在 PySpark 端预先对数据按照多维属性组合 (`event_type`, `category_level1`, `brand`) 进行 GroupBy 预聚合，生成包含全维度交叉统计特征的立方体文件 (Dashboard Cube)。后端收到多属性联合筛选的 API 请求后，直接对预计算的立方体进行定位并执行流式检索，毫秒级快速匹配并返回结果，避开了明细行扫描。
2. **第二阶段：明细数据扫描兜底**——若由于硬件故障或路径异常导致多维立方体缓存文件缺失或解析损坏，缓存系统会自动触发降级兜底。后端会直接读取清洗后的明细事件表，在本地内存中对数据进行多属性联合过滤，并重新计算去重后的独立用户数与独立会话数。这确保了在计算文件异常时系统仍具有高可用性，但查询耗时会退化至秒级。

2.3.2 React 前端与 Ops 证据面板

2.3.2.1 TanStack Query 生命周期托管

本系统的用户界面使用 React 与 Vite 构建，利用 TanStack Query 对数据交互请求进行生命周期托管，利用 ECharts 渲染关系图、转化漏斗、时序波动等复杂图表。前后端通过定义的路由和代理机制保持松耦合。前端页面按照分析主题组织，包含 Dashboard、Behavior、Conversion 等多个独立页面。

2.3.2.2 BenchmarkEvidenceService 证据链路

为了向开发与运维人员展示大数据处理流水线的物理执行细节，项目设计了专门的运维 (Ops) 证据看板。传统的 Web 看板通常隐蔽了后端基础设施的存在，而本项目的 Ops 看板能够将底层的 YARN Application 状态、Spark History 执行画像以及 HDFS 数据读写信息进行前台闭环展示。

后端的 BenchmarkEvidenceService 提取 SQLite 和运行 Manifest 中的数据血缘证据，同时调用 Python-Hadoop API 以及 Spark History Server REST API，采集物理执行参数。这些参数包括每个 Stage 的运行耗时、网络 Shuffle 读写字节数、任务失败与重试率以及 Executor 内存溢写字节数。前端 Ops 页通过拉取这些证据，在单一界面中展现端到端的数据流动关系，帮助评审人员快速验证报告中实验结论的可信度。

第 3 章 需求分析与实验对照设计

3.1 系统总体目标与工程验收指标

系统的总体目标是构建一套可运行、可复现、可对比的电商行为大数据分析实验平台。平台既要能处理真实 Kaggle 电商用户行为数据，又要适合本机 Docker 容器环境，不能为了追求形式上的“分布式集群”而频繁触发物理内存 OOM 或占满磁盘空间。最终的工程交付应包含四类成果：第一，支持自适应降级的 ARM64 Docker Hadoop YARN 实验集群；第二，基于 PySpark 的分布式数据分析 Pipeline 与全链路内存优化控制实现；第三，1% 和 5% 多梯度样本下的性能 Benchmark 对照评估证据；第四，前端 Ops 运维证据面板和 LaTeX 报告中的实验图表。

在工程验收指标上，系统必须通过以下硬性门槛：首先，执行 `docker compose --profile yarn-lab up -d` 后，NameNode、ResourceManager、NodeManager、Spark Client 和 Spark History Server 等集群基础容器能够正常拉起；其次，`start-dev.sh` 脚本能一键在宿主机 5051 和 5173 端口启动 Flask 后端与 Vite 前端，并正确关闭占用冲突端口的陈旧进程；第三，Spark 作业的提交能够顺利生成 SUCCEEDED application；最后，前端 Ops 面板能够完整显示每次运行的输入快照、YARN application id、质量状态以及多维性能 Benchmark 数据。

在实验验收指标上，系统需要对每个模块的运行状态进行定量评估。记录的指标包括纯计算耗时 (Elapsed Seconds)、数据吞吐率 (Cleaned Rows/Sec)、输入输出行数、Driver 侧 RSS 物理内存峰值、Executor 侧 Shuffle 读写与 Spill 磁盘溢写字节数、任务失败/重试次数以及质量门禁通过状态。其中最关键的性能结论指标如下：

1. 无论何种数据规模，Driver 进程均不会因为明细数据的 `collect()` 收集操作或大体积 JSON 的载入而触发 OOM 崩溃。
2. 商品推荐、用户留存等计算模块在 Executor 侧执行时，其明细收集行数必须固定在预览阈值 (Preview Limit) 内，质量指标的统计分析改由 Spark 聚合算子分布式处理。
3. 关联分析中的候选 Key 组合数必须受控，不能出现失控的二次级组合膨胀。
4. 5% 样本相较 1% 样本的整体 Pipeline 运行时间增长应接近线性，无因执行计划不合理导致的长尾 Task 拖延。
5. Spark History Server 能够正常查看 event log 物理计划，通过限制执行计划 plan 字符串的保存长度，控制日志体积不触发 History 服务的处理崩溃。

3.2 系统功能与非功能性需求分析

在系统功能需求方面，数据处理链路需要实现从原始半结构化 CSV 事件流到分析层指标缓存的完整闭环。原始数据字段包含 `event_time`、`event_type`、`product_id`、`category_id`、`category_code`、`brand`、`price`、`user_id` 和 `user_session`。系统需要分布式计算会话事实表、多维指标看板立方体、日趋势、转化漏斗、商品推荐、关联图谱、异常信号、留存矩阵、销量预测以及多触点收入归因等模块。集群运行功能需要支持 Spark 提交流水线在 YARN client mode 下的稳定分配，各组件 UI 必须使用偏移后的安全端口以规避本地端口冲突。运维展示功能则要求 React 前端不仅能展示业务数据，还需支持手动触发异步 Spark 作业刷新、查询 SQLite 作业元数据并实时跟踪清洗质量门禁状态。

在非功能需求方面，首先，系统在容器镜像构建上必须具备高度的架构兼容性。针对 ARM Mac 设备，所有 Hadoop 与 Spark 底层基础镜像必须使用 ARM64 架构进行编译，拒绝下载 amd64 镜像触发跨架构模拟带来的超额性能损耗。其次，内存稳定性是本系统的核心考量。系统并不承诺任意无界规模数据下均不出现内存问题，而是要求在受限物理资源下合理控制 Shuffle partition、在 DataFrame 链式聚合时于合适位置 persist 指标并在结束后 unpersist，将 driver 收集与 JSON 构建限制在有界 summary 中。最后，证据可追溯性 (Lineage & Provenance) 要求每次计算作业均需将当前的配置快照、输入文件校验和、清理策略和 YARN application id 持久化写入数据库与运行 Manifest，由 Flask 指标检索服务进行血缘一致性证明，拒绝任何人工硬编码数据的混入。

3.3 1% 与 5% 多配置梯度实验对照设计

为深入定量评估系统在分布式环境下的性能特征、网络 Shuffle 开销以及调优机制的实际增益，本项目在需求分析阶段设计了多配置梯度实验。其中，1% 样本用于执行五组多配置对照实验，以此排查性能瓶颈并验证特定优化算子的作用；5% 样本则用于执行大压力极限跑通，用以评估系统在数据量扩大 5 倍时的可扩展性。

实验对照组的详细参数设计如表3.1所示：

通过这一系列对照组的设置，我们能够剥离出不同物理层级的性能贡献：

- **YARN 分布式开销评估：**通过对比 Baseline local 与 YARN-only CSV，我们可以量化在单机多容器环境下，因加入 HDFS 分布式存储读取以及 YARN ResourceManager/NodeManager 容器资源申请和序列化所引入的基础开销。

表 3.1 多配置对照实验设计矩阵

对照组名称	计算核数	数据源	开启 AQE	算法限流	实验验证目的
Baseline local	4 核	本地 CSV	否	否	评估本地 Spark 的纯单机基准性能。
YARN-only CSV	1 实例	HDFS CSV	否	否	评估仅迁移至 YARN 时的网络与分布式开销。
YARN + AQE CSV	1 实例	HDFS CSV	是	否	评估自适应查询执行对物理 Shuffle 的调优。
YARN + AQE + Algo	1 实例	HDFS CSV	是	是	评估 Driver 侧数据限流与组合防膨胀的收益。
YARN + Parquet	1 实例	HDFS Parquet	是	是	评估列式存储布局对于 I/O 和过滤的增益。

- **执行计划优化评估:** 对比 YARN-only CSV 和 YARN + AQE CSV, 能够证明仅通过增加分布式节点并不会自然带来性能提升; 在倾斜数据下, AQE 优化器的分区合并与数据倾斜 Join 处理对于整体耗时下降具有决定性作用。
- **算法与内存优化评估:** 通过第四组实验, 验证在 Driver 侧设立数据预览阈值、在推荐模块中限制排序集大小, 以及限制 SQL plan 长度对控制内存泄漏与避免 OOM 的实际价值。
- **存储布局评估:** 通过 YARN + Parquet 对照组, 量化相较于纯文本 CSV, Parquet 压缩、谓词下推与列裁剪对物理 I/O 和清洗流水线执行的加速比。

3.4 数据清理与存储安全需求

在大数据实验中, 计算任务执行产生的 event log、临时中间结果、清洗后 Parquet 目录以及旧的 Benchmark 数据会快速膨胀, 占满 Docker 虚拟磁盘和 HDFS 的 Block 块存储, 导致 NameNode 无法分配新的块而导致后续提报作业失败。因此, 系统必须具备严格的数据清理需求。

清理策略规定：系统需保留原始 **Kaggle** 电商 CSV 文件与 1%/5% 本地样本以供数据可溯源性校验；保留最终完成的 6 个 **Spark History event log** 供页面展示；而对于在此之前的测试、失败重试、中间临时输出以及冗余的 **application log**，必须在每次新 **Benchmark** 任务启动前由清理脚本执行物理删除。这既能维持课程实验展示证据的完整性，又可将宿主机 **Docker** 磁盘总占用控制在安全水位内。

第 4 章 系统架构与数据工程实现

4.1 总体架构与环境配置

系统在整体拓扑上采用”数据层、计算层、服务层、展示层”四层架构。数据层包括本地 Kaggle 原始 CSV、本地 1%/5% 样本、HDFS CSV 输入和 HDFS Parquet 清洗层；计算层包括 Spark SQL pipeline、AQE、推荐/关联/异常等算法模块和 Spark History event log；服务层包括 Flask API、SQLite 作业记录、run manifest 与 benchmark evidence 服务；展示层包括 React 前端业务页面、质量页和 Ops 页。

图4.1展示了 Docker YARN 实验集群拓扑。该拓扑保留了 Hadoop 课程设计所需的 NameNode、DataNode、ResourceManager、NodeManager 和 Spark History Server，同时增加 spark-client 作为提交入口。前端与后端按 `start-dev.sh` 启动：Flask 后端为 5051，Vite 前端为 5173；HDFS UI、YARN UI 和 Spark History UI 分别映射到 19870、18088 和 28080，避免与宿主机已有 master 容器冲突。

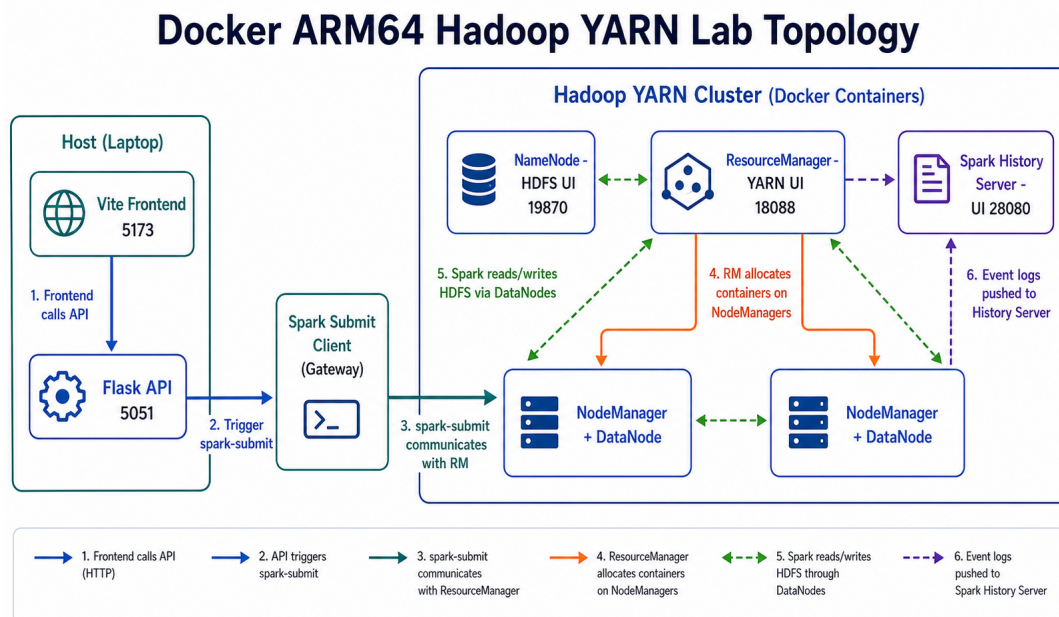


图 4.1 Docker ARM64 Hadoop YARN 实验集群拓扑（由 chat-gpt-image-2 生成）

在环境安装与启动上，本项目自建精简 ARM64 Hadoop 镜像，只包含运行 Hadoop 所需的 Java、Hadoop 文件、配置和启动脚本。与 `docker commit master` 相比，该方法不会复制 HDFS 数据目录、日志、Hive、HBase 和临时文件，因此镜像体积更可控，也符合 ARM Mac 的架构约束。集群通过 `yarn-lab profile` 启动，端口规划如表4.1所示。系统保留两套演示入口。答辩现场如果重点展示前端页面和业

务分析结果，推荐使用本地开发模式，因为它复用宿主机 Python 虚拟环境和 Vite，启动更快，端口也与前端代理一致。如果需要展示 Hadoop/YARN 技术栈，则启动 yarn-lab profile，再打开相关 UI，此时前端通过启动脚本指向 127.0.0.1:5050 代理。样本数据准备脚本会从本地用户级样本准备 1% 和 5% 输入，再上传到 HDFS 对应目录，清理后的 HDFS 关键路径仅保留正式实验输入与 6 个最终的 Spark History event log。

表 4.1 实验环境端口规划

服务	宿主机端口	说明
Flask API	5051	start-dev.sh 启动的后端接口，前端通过该地址访问 /api/v1。
Vite 前端	5173	start-dev.sh 启动的正确前端访问地址，用于打开 /ops。
YARN Flask API	5050	web-yarn 容器的宿主机端口映射，内部仍监听 5051。
NameNode UI	19870	查看 HDFS 文件、block 和 DataNode 状态。
ResourceManager UI	18088	查看 YARN application、队列和运行状态。
Spark History Server	28080	查看 Spark event log 和 stage/job 信息。

4.2 分布式提交链路与内存优化控制

4.2.1 YARN Client 提交流程

系统第一阶段默认采用 YARN client mode，因为 Flask 与 React 仍读取 spark-client 容器中的 data/cache。提交前先将 spark_jobs 打包为 zip，再通过 spark-submit 提交到 YARN。脚本内部会设置 Hadoop 配置目录、Spark driver host、Python 解释器路径和 event log 参数，然后调用 spark-submit。如果用户从前端点击刷新，Flask 会创建 SQLite 作业记录，再由 SparkPipelineRunner 调用提交脚本。运行完成后，SparkPipelineRunner 从 stdout 中解析 manifest 路径并写入 SQLite。为避免后端消息过长，runner 只保留 stdout/stderr tail，完整日志写入 run_logs。

4.2.1.1 Event Log 压缩治理

针对 event log 体积过大导致 History Server 读取解析缓慢或崩溃的漏洞，本项目在配置中限制了逻辑执行计划 plan 字符串的保存长度：

```
spark:
```

```
master: yarn
deployMode: client
appName: ecommerce-yarn-lab
config:
  spark.sql.adaptive.enabled: true
  spark.sql.adaptive.skewJoin.enabled: true
  spark.sql.shuffle.partitions: 64
  spark.sql.maxPlanStringLength: 8192
  spark.eventLog.enabled: true
  spark.eventLog.dir: hdfs:///spark-history
```

通过 `spark.sql.maxPlanStringLength` 将计划树长度硬性裁剪至 8192，极大地降低了 event log 的体积，并彻底消除了 History Server 端的接口读取 500 风险。此外，项目同时提供了 `yarn-cluster.yaml` 和 `submit_yarn_cluster.sh` 作为后续扩展入口。提交脚本通过环境变量显式设置 Hadoop 配置目录、Spark driver host、Python 解释器路径和 event log 参数，其中 `--conf spark.sql.maxPlanStringLength=8192` 是压缩 event log 体积的关键开关。

4.2.2 Driver 端内存保护

大批量分析计算中最核心的瓶颈在于防止 Spark Driver 发生内存溢出 (OOM)。本项目对此制定了四向优化控制：

4.2.2.1 全局收集禁用与预览限流

早期推荐等模块会直接把完整的结果集通过 `collect()` 收集回 Driver，这在大样本下构成了严重的单点瓶颈。优化后的设计仅允许收集特定展现上限（例如 `preview_limit = 1000`）的行数用于前台渲染，而完整的数据指标则直接利用并行算子在分布式 Executor 节点聚合，并以 Parquet 形式直接落入分布式文件系统：

```
preview_rows = (
    recommendation_features
    .orderBy(F.desc("score"))
    .limit(preview_limit)
    .collect()
)
quality = recommendation_features.agg(
    F.count("*").alias("candidate_rows"),
    F.avg("confidence").alias("avg_confidence"),
    F.sum(F.col("is_fallback").cast("int")).alias("fallback_rows"),
)
```

4.2.2.2 未分区 WindowExec 消除

Spark 执行计划分析表明，未声明 `partitionBy()` 的窗口函数（例如计算全局排序）会强行触发全局重分区，将百万级事件流全部 Shuffle 到单个物理 Partition

的单 Task 执行，形成严重的执行长尾甚至引发内存耗尽。项目通过引入开窗分区键规范，在所有排序场景中强制要求利用会话 ID、用户 ID 或日期进行局部哈希，从物理层杜绝了单节点长尾 Task。

4.2.2.3 会话级组合膨胀防御

在进行商品关联分析等配对计算时，若单个会话（Session）包含的商品数过多，在进行自连接（Self-Join）时会产生高达 $O(n^2)$ 级别的笛卡尔积行数爆炸。系统在 `spark_jobs/affinity.py` 中设立了严格的 session 长度安全边界机制：

```
session_lengths = cleaned_df.groupBy("user_session").count()
eligible_sessions = session_lengths.filter(F.col("count") <= max_pair_products_per_session)
joined_pairs = cleaned_df.join(eligible_sessions, "user_session", "left_semi")
```

通过限制单个会话的最大处理商品数，系统成功地将 co-occurrence 的笛卡尔积膨胀系数控制在受托范围。

4.3 数据清洗与质量门禁

4.3.1 字段标准化与类型转换

本系统的计算入口是 `spark_jobs/main.py` 中的 `run_job`。原始事件以严格的元数据 Schema 进行加载。数据清洗阶段核心完成四件事：

1. 利用 Spark 时间函数将非标字符串 `event_time` 转换成 `Timestamp` 类型 `event_timestamp`，并派生 `event_date` 与 `event_hour`。
2. 对 `brand`、`user_session` 和 `category_code` 执行空值判断，缺失部分统一使用 "unknown" 填充，利用字符串切分函数将 `category_code` 拆分出一级与二级类目列。
3. 过滤掉关键字段缺失的行，并通过硬编码门限排除价格不在区间 `[0, 100000]` 内的异常销售额数据。
4. 对五属性事件主键进行物理排重，主键字段为 `event_time`、`event_type`、`product_id`、`user_id` 和 `user_session`。

```
cleaned = (
    normalized
    .filter(F.col("event_type").isin(EVENT_TYPES))
    .filter(F.col("event_timestamp").isNotNull())
    .filter(F.col("product_id").isNotNull())
    .filter(F.col("user_id").isNotNull())
    .filter(F.col("price").isNull() | ((F.col("price") >= 0) & (F.col("price") <= 100000)))
    .dropDuplicates(["event_time", "event_type", "product_id", "user_id", "user_session"])
)
```

4.3.1.1 自适应 3-Sigma 异常会话拦截

在数据过滤阶段，项目针对高频爬虫与异常刷单数据引入了自适应 3-Sigma 动态风控拦截机制。该机制以会话为单位，动态统计事件发生密度 D_s 以及会话总购买笔数 P_s ，计算全局均值 μ 与标准差 σ ，并设立防御警戒线：

$$D_{\text{limit}} = \min(10.0, \max(2.0, \mu_D + 3\sigma_D)) \quad (4.1)$$

$$P_{\text{limit}} = \min(15.0, \max(5.0, \mu_P + 3\sigma_P)) \quad (4.2)$$

凡是满足 $D_s > D_{\text{limit}}$ （参见公式（4.1））或 $P_s > P_{\text{limit}}$ （参见公式（4.2））的异常会话被判定为 bot 流量，利用 left_anti 关联进行彻底剔除，实现下游分析指标的纯净化。

4.3.1.2 质量门禁评估机制

清洗完成后的数据集会经过质量门禁评估，包括行数对比（输入行数与输出行数的差异比例）、关键字段空值率检查和价格分布异常值检测。质量门禁的评估结果被记录到运行 manifest 中，作为后续 Ops 证据看板的数据来源。

4.4 会话与特征工程

4.4.1 会话事实表与转化漏斗

转化漏斗不能单纯地对全量事件分组，而必须还原用户的真实会话轨迹。系统通过 groupBy("user_session") 聚合每个会话的首次行为，构建会话事实表。

4.4.1.1 合法行为路径判定

为了拦截分布式数据流转中因时序错乱带来的无效转化，在此定义了合法行为路径的逻辑表达式：

$$\text{ValidCartPath} = 1 \iff t_{\text{view}} \leq t_{\text{cart}} \quad (4.3)$$

$$\text{ValidCartPurchasePath} = 1 \iff t_{\text{view}} \leq t_{\text{cart}} \leq t_{\text{purchase}} \quad (4.4)$$

若系统检测到 $t_{\text{purchase}} < t_{\text{view}}$ ，则说明数据存在时序异常，会话将被标为时序异常会话，不计入标准正向转化中。

```
session_facts = (
    df.groupBy("user_session")
      .agg(
        F.min(F.when(F.col("event_type") == "view", F.col("event_timestamp"))).alias("first_
```

```

    ↪ view_ts"),
      F.min(F.when(F.col("event_type") == "cart", F.col("event_timestamp"))).alias("first_
    ↪ cart_ts"),
      F.min(F.when(F.col("event_type") == "purchase", F.col("event_timestamp"))).alias("
    ↪ first_purchase_ts"),
      F.sum(F.when(F.col("event_type") == "purchase", F.coalesce(F.col("price"), F.lit(0))
    ↪ ).otherwise(0)).alias("session_revenue"),
    )
  .withColumn("valid_cart_path", F.col("first_cart_ts") >= F.col("first_view_ts"))
  .withColumn("valid_cart_purchase_path", F.col("first_purchase_ts") >= F.col("first_cart_
    ↪ ts"))
)

```

4.4.1.2 商品级转化率计算

商品级转化进一步把粒度设为 `product_id + user_session`，分别统计浏览、加购及购买会话数，并输出各自的转化率。转化率定义如公式（4.5）所示：

$$CR_{\text{product}} = \frac{\text{PurchaseSessions}}{\text{ViewSessions}} \quad (4.5)$$

4.4.2 高频行为路径与转移矩阵

高频行为路径聚焦于用户在单一会话内的时序走向。系统使用 Spark 的 `collect_list` 算子收集会话事件结构体，按时间戳升序重排，并把前 8 个事件合并拼接为路径签名。同时，将路径中相邻的行为转换为转移边，生成状态转移频次矩阵。

```

session_paths = (
  cleaned_df.groupBy("user_session")
    .agg(F.sort_array(F.collect_list(event_struct)).alias("event_structs"))
    .withColumn("events", F.expr("transform(event_structs, x -> x.event_type)"))
    .withColumn("path_signature", F.concat_ws(" -> ", F.expr("slice(events, 1, 8)")))
)
transitions = (
  session_paths
    .select(F.expr("arrays_zip(slice(events, 1, size(events)-1), slice(events, 2, size(
    ↪ events)-1))").alias("pairs"))
    .select(F.explode("pairs").alias("pair"))
    .groupBy(F.col("pair.0").alias("from_event"), F.col("pair.1").alias("to_event"))
    .count()
)

```

转移矩阵和路径漏斗被前台渲染为桑基图（Sankey Diagram），直观展示流失率。

4.4.3 双粒度特征集市

特征集市为下游异常监控和生命周期管理提供统一的数据准备。项目在 PySpark 侧构建了两个核心维度的日级别特征表：

4.4.3.1 商品日特征

以 `date + product_id + brand` 为键，聚合单日浏览量、购买量、GMV、均价和转化指标。商品日特征表是推荐引擎与运营优化模块的核心输入。

4.4.3.2 用户日特征

以 `date + user_id` 为键，汇总单日访问时长、加购数、购买数、消费总额和偏好类目。用户日特征表为生命周期分群与个性化推荐提供行为画像基础。

4.4.3.3 新鲜度延迟监控

为了实现对流式写入延迟的自动化校验，引入新鲜度延迟计算，其定义为当前系统时间与数据中最大事件时间戳的差值 $\Delta T = T_{\text{system}} - \max(T_{\text{event}})$ ，同时通过回窗分析过滤 `late arrival` 迟到日志，确保计算特征的新鲜度始终位于健康水平。

第 5 章 业务分析与智能决策模块实现

5.1 深度业务预测与需求估计

5.1.1 双尺度需求预测与 WAPE 精度度量

需求预测模块 (`forecasting.py`) 实现了全站大盘与热门分类的双尺度未来 7 天销量预测。为避免引入外部深度学习运行时的复杂性,项目在 Driver 侧基于 Pure NumPy 独立实现了一个自回归多层感知机 (Global AR-MLP) 时序模型。

5.1.1.1 16 维特征向量与 GlobalAR-MLP 模型

神经网络模型在隐藏层采用 ReLU 激活函数,输出层采用线性激活。模型输入的特征向量为 16 维,其组成如下:

$$X_t = [G_{t-1}, G_{t-2}, G_{t-3}, G_{t-7}, O_{t-1}, O_{t-2}, O_{t-3}, O_{t-7}, W_0, W_1, W_2, W_3, W_4, W_5, W_6, \gamma] \quad (5.1)$$

其中, G 为过去滑窗的销售额 (GMV), O 为历史订单数, W 为星期周期的 7 维 One-Hot 编码, γ 为外生协变量修正因子 (反映目标日浏览量与历史平均浏览量的比例,作为流量的趋势传导)。

5.1.1.2 前向传播与动量 SGD 训练

神经网络的前向传播逻辑如下:

$$h = \max(0, X \cdot W_1 + b_1) \quad (5.2)$$

$$\hat{Y} = h \cdot W_2 + b_2 \quad (5.3)$$

其中,权重矩阵采用 Kaiming Normal (He 初始化) 方式实例化以保障深层传导,反向传播采用基于均方误差 (MSE) 梯度的动量 SGD (Momentum SGD) 优化算法更新参数:

$$v_W \leftarrow \beta v_W + \eta \nabla_W \mathcal{L} \quad (5.4)$$

$$W \leftarrow W - v_W \quad (5.5)$$

其中, $\beta = 0.9$ 为动量因子,初始学习率 $\eta = 0.05$,并采用指数衰减调度策略 (每 20 个 epoch 乘以衰减因子 0.95) 以平衡收敛速度与稳定性。

5.1.1.3 滑动多步预测与层次时序协调

在预测未来的 7 天时，采用滑动多步预测（Sliding-window Multi-step Forecasting）流程：将前一天生成的预测值作为下一天输入的 lag1 项滚入特征向量，实现迭代预测。

模型采用加权绝对百分比误差（WAPE, Weighted Absolute Percentage Error）进行回测评估精度：

$$\text{WAPE} = \frac{\sum_d |Y_d - \hat{Y}_d|}{\sum_d Y_d} \quad (5.6)$$

对于独立的实体时序预测结果，大盘预测（自上而下）与各大品类预测（自下而上）会出现数学上的不一致。系统执行层次时序协调对齐：首先计算大盘与所有类目预测之和的偏差 $\delta = \hat{Y}_{\text{site}} - \sum_c \hat{Y}_c$ ，然后依据历史 28 天各大品类在对应指标（GMV 或销量）上的归一化均值比率作为权重 w_c ：

$$w_c = \frac{\bar{Y}_{c,\text{hist}}}{\sum_{c'} \bar{Y}_{c',\text{hist}}} \quad (5.7)$$

其中 $\bar{Y}_{c,\text{hist}}$ 为品类 c 在历史 28 天内对应评估指标（GMV 或销量）的平均值。系统将偏差加权分配给各个品类以确保层级对齐：

$$\hat{Y}'_c = \hat{Y}_c + w_c \cdot \delta \quad (5.8)$$

若历史样本天数太少（少于 7 天），模型将自动降级为滚动均值平滑回归算法。

5.1.2 商品关联图谱挖掘与置信度评价

关联分析（spark_jobs/affinity.py）旨在识别会话内频繁共同加购或购买的商品对。

5.1.2.1 支持度-置信度-Lift 评价体系

在经过会话长度拦截器过滤后，系统构建商品共现矩阵。对任意商品对 (A, B) ，其核心评价指标定义如下：

$$\text{Support}(A, B) = \text{Sessions}(A \cap B) \quad (5.9)$$

Support 为绝对共现频次，表示同时出现在 A 与 B 中的独立会话数，未做归一化处理。

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A, B)}{\text{Support}(A)} \quad (5.10)$$

$$\text{Lift}(A, B) = \frac{\text{Support}(A, B) \cdot N_{\text{total}}}{\text{Support}(A) \cdot \text{Support}(B)} \quad (5.11)$$

其中 N_{total} 为样本中的总独立会话数。由于支持度使用绝对频次定义，Lift 公式需要乘以 N_{total} 以反映共现概率与边缘概率乘积的比值。

5.1.2.2 共现图谱与会话长度防御

关联结果以前端力导向关系图 (Force-Directed Graph) 展示，并严格记录共现产生的特征行数以防内存组合爆炸。会话长度拦截器 (Session Length Guard) 以会话内不同商品数 (distinct product_count) 为守卫指标，过滤掉商品数超过阈值 (默认 ≤ 20) 的会话后再执行自连接，从源头将共现组合的笛卡尔积规模控制在安全范围内，防止极端长会话导致的 $O(n^2)$ 共现矩阵爆炸。

5.2 用户生命周期与留存分析

5.2.1 Cohort 留存分析与购物车挽回

5.2.1.1 Cohort 留存分析与复购价值曲线

5.2.1.2 月级步长计算

留存分析仅提取购买事件，寻找每个用户的首次消费月份作为 Cohort 分群标识，并按月级步长计算其活跃生存期。系统分别统计活跃留存率与复购留存率：

$$\text{RetentionRate}_{c,p} = \frac{\text{ActiveUsers}_{c,p}}{\text{CohortUsers}_c} \quad (5.12)$$

其中 c 为 Cohort 标识 (首购月份)， p 为距首购的月级步长。

5.2.1.3 累计生命周期价值

复购价值曲线则在留存矩阵的纵轴上执行窗口累加，计算累计生命周期价值 (LTV)，为用户长期价值提供科学研判：

$$\text{LTV}_{c,p} = \sum_{k=0}^p \frac{\text{Revenue}_{c,k}}{\text{PurchaseUsers}_{c,k}} \quad (5.13)$$

其中， $\text{PurchaseUsers}_{c,k}$ 为 Cohort c 在第 k 个周期内实际产生购买行为的用户数 (非 Cohort 总人数)，从而使 LTV 的累积含义为“平均每个活跃购买用户的累计消费额”，而非“平均每个留存用户的累计消费额”。

5.2.2 RFM 生命周期分群模型

客户生命周期模块(`spark_jobs/lifecycle.py`)基于 RFM(Recency, Frequency, Monetary) 分析框架,对全量用户进行自动化分群与风险评级。其目标是为下游的推荐个性化、实验分组和运营决策提供用户级的行为画像标签。模块采用确定性规则引擎而非聚类算法,以确保分群结果在每次运行中完全可复现,便于实验对照与策略审计。

5.2.2.1 三维评分体系(近因-频率-消费力)

系统对每个用户在全量历史数据上聚合三个核心维度的评分:

Recency (近因度)。以数据快照中最新日期 $d_{\max}$ 为基准,计算用户最近一次活跃距今的天数 R :

$$R = d_{\max} - d_{\text{last_active}} \quad (5.14)$$

评分规则为:

$$S_R = \begin{cases} 3 & R \leq 1 \\ 2 & R \leq 7 \\ 1 & R > 7 \end{cases} \quad (5.15)$$

Frequency (频率度)。统计用户在不同日期产生购买行为的天数 F_{days} :

$$S_F = \begin{cases} 3 & F_{\text{days}} \geq 2 \\ 2 & \text{purchases} > 0 \\ 1 & \text{otherwise} \end{cases} \quad (5.16)$$

Monetary (消费力)。统计用户历史累计消费金额 M :

$$S_M = \begin{cases} 3 & M \geq 500 \\ 2 & M > 0 \\ 1 & M = 0 \end{cases} \quad (5.17)$$

5.2.2.2 七段确定性分类

在 RFM 评分的基础上,系统通过优先级规则链将用户分为七个生命周期段:

```
.withColumn("lifecycle_segment",
  F.when(
    (F.col("revenue") >= 1000.0) & (F.col("purchase_days") >= 2),
    F.lit("champion"))
  .when(
```

```
(F.col("revenue") >= 500.0) & (F.col("purchases") > 0),
  F.lit("high_value"))
.when(F.col("purchase_days") >= 2, F.lit("loyal"))
.when(F.col("purchases") > 0, F.lit("buyer"))
.when(F.col("carts") > 0, F.lit("cart_intent"))
.when(F.col("views") > 0, F.lit("browser"))
.otherwise(F.lit("unknown"))
```

七段分类按优先级从高到低依次为：**Champion**（冠军用户，累计消费 ≥ 1000 且多日复购）、**High Value**（高价值用户，消费 ≥ 500 且有购买记录）、**Loyal**（忠诚用户，跨日购买天数 ≥ 2 ）、**Buyer**（普通购买者，有至少一次购买）、**Cart Intent**（加购意向，仅有加购行为）、**Browser**（纯浏览者，仅有浏览行为）、**Unknown**（无有效行为数据）。该分类采用规则链的短路求值逻辑，每个用户仅匹配第一个满足条件的段，确保分群结果的互斥性与完备性。

5.2.2.3 风险带评级与建议动作

风险带（Risk Band）是面向运营干预的二级标签，用于标识用户的流失风险与转化机会：

$$\text{RiskBand} = \begin{cases} \text{at_risk} & R \geq 14 \\ \text{convert_intent} & \text{purchases} = 0 \wedge \text{carts} > 0 \\ \text{active_value} & \text{purchases} > 0 \\ \text{observe} & \text{otherwise} \end{cases} \quad (5.18)$$

其中，**at_risk** 表示用户已超过 14 天未活跃，存在流失风险；**convert_intent** 表示有加购但未购买，存在转化机会；**active_value** 表示有购买行为的活跃用户；**observe** 表示低优先级的观察对象。每个风险带自动匹配推荐动作，如 **at_risk** 用户建议“通过品类个性化优惠券进行再激活”，**convert_intent** 用户建议“优先处理购物车恢复并排查价格或库存摩擦”。

生命周期模块的输入为用户级别的日聚合特征表（**daily_user**）和品类日聚合特征表（**daily_category**），输出的用户分群标签（**lifecycle_segment**、**risk_band**）是下游实验平台与推荐系统的核心分组变量。风险带标签直接流入决策引擎（**decision_maker.py**），用于触发差异化的运营干预策略。

5.2.2.4 购物车漏斗挽回与优先级评分机制

5.2.2.5 放弃判定逻辑

购物车挽回模块针对加购而未购买的用户行为进行评估。在会话-商品粒度下，若用户执行了加购却在会话期内无对应的购买记录，判定为放弃；若购买时间晚于加购时间，判定为成功挽回。

5.2.2.6 优先级评分公式

系统依据商品的流失量级与转化特征，设计了用于排程挽回短信或优惠券的挽回优先级评分机制：

$$\text{PriorityScore}_i = \text{AbandonedValue}_i \times \text{AbandonmentRate}_i \quad (5.19)$$

其中， AbandonmentRate_i 为商品 i 的放弃率（含用户主动移除场景），该指标比 $(1 - \text{RecoveryRate})$ 更精确，因为存在用户主动移除购物车商品（`explicit_remove`）这一独立状态。优先级高的商品将被推入挽回队列的前列。

在商品级挽回动作之外，系统还在品类维度上定义了三种类目级运营动作：当品类移除率过高时触发 `category_merchandising_review`（品类陈列审查）；当品类放弃率超过阈值时触发 `category_recovery_campaign`（品类级挽回活动）；其他情况标记为 `category_watch`（品类观察），帮助运营团队从品类结构层面排查流失根因。

5.3 归因、优化与推荐决策

5.3.1 收入归因与运营优化

5.3.1.1 多触点收入归因

多触点收入归因模块（`attribution.py`）将用户的消费额科学地分摊给在购买事件前发生的各个交互触点（Views, Carts）。

5.3.1.2 Linear 均分模型

将购买总额 R 均匀分配给会话内的 N 个触点：

$$CR(t_k) = \frac{R}{N} \quad (5.20)$$

5.3.1.3 Time-Decay 时间衰减模型

依据触点距离购买发生的时间距离进行指数级分配。权重随逆时序反向递减：

$$W(t_k) = d^{N-k} \quad (5.21)$$

$$CR(t_k) = R \times \frac{W(t_k)}{\sum_{j=1}^N W(t_j)} \quad (5.22)$$

其中，时间衰减常数设为 $d = 0.7$ 。在 Spark 侧，该算法通过开窗排序与列矩阵相乘算子实现全分布式的高吞吐运行，代码实现口径如下：

```
weighted = (
    joined
    .withColumn("touch_count", F.count("*").over(by_purchase))
    .withColumn("position_before_purchase", F.row_number().over(asc))
    .withColumn("reverse_position_before_purchase", F.row_number().over(desc))
    .withColumn("linear_credit", F.col("purchase_revenue") / F.col("touch_count"))
    .withColumn("first_touch_credit", F.when(F.col("position_before_purchase") == 1, F.col("
    ↪ purchase_revenue")).otherwise(0.0))
    .withColumn("last_touch_credit", F.when(F.col("reverse_position_before_purchase") == 1,
    ↪ F.col("purchase_revenue")).otherwise(0.0))
    .withColumn("time_decay_weight", F.pow(F.lit(decay_base), F.col("reverse_position_before
    ↪ _purchase") - 1))
    .withColumn("time_decay_credit", F.col("purchase_revenue") * F.col("time_decay_weight")
    ↪ / F.sum("time_decay_weight").over(by_purchase))
)
```

5.3.1.4 双层稳健化运营优化

运营优化决策模块 (optimization.py) 旨在受限营销预算和推荐位容量下，为候选商品寻找最佳的促销手段以最大化增量收益。

5.3.1.5 Bayes 先验平滑

为规避冷启动商品因小样本带来的高估误差，候选商品的购买率通过先验平滑收缩购买率 (Bayes Shrunk Rate) 进行第一层稳健化处理：

$$\theta_i = \frac{P_i + \alpha \cdot \bar{P}_{\text{global}}}{V_i + \alpha} \quad (5.23)$$

其中， P_i, V_i 分别为商品的购买量与浏览量， $\bar{P}_{\text{global}}$ 为全站平均转化率，先验置信权重常数设为 $\alpha = 50$ 。

5.3.1.6 Wilson 置信下界

第二层稳健化采用置信下界度量（Wilson Score Interval Lower Bound），对商品的真实转化概率给出保守的下界估计：

$$\theta_i^{\text{wilson}} = \frac{\hat{p}_i + \frac{z^2}{2V_i} - z\sqrt{\frac{\hat{p}_i(1-\hat{p}_i)}{V_i} + \frac{z^2}{4V_i^2}}}{1 + \frac{z^2}{V_i}} \quad (5.24)$$

其中 $\hat{p}_i = P_i/V_i$ 为商品 i 的经验购买率，置信水平选定为 95%（即 $z = 1.96$ ），以此保守估计该商品的转化底线。

为了综合评估样本规模和置信下界表现，系统进一步定义了商品的置信度权重 c_i ：

$$c_i = \min\left(1.0, \sqrt{\frac{V_i}{500}}\right) \times \left(0.5 + 0.5 \times \min\left(1.0, \frac{\theta_i^{\text{wilson}}}{\bar{P}_{\text{global}}}\right)\right) \quad (5.25)$$

该权重既惩罚低曝光商品的置信度（通过样本量折价），又拉近了低转化商品的权重。并以此计算风险偏离度（或称风险得分） $R_i = 1 - c_i$ 。

5.3.1.7 MILP 数学模型与 Gurobi/贪心求解

随后，系统构建了混合整数线性规划（MILP）数学决策模型：

$$\max_{x_{i,a}} \sum_{i \in C} \sum_{a \in A} x_{i,a} \cdot [\text{BaselineGMV}_i \cdot \Delta_a \cdot c_i - \lambda \cdot R_i \cdot \text{BaselineGMV}_i \cdot \Delta_a \cdot c_i] \quad (5.26)$$

其中， $\text{BaselineGMV}_i = V_i \cdot \theta_i \cdot \bar{p}_i$ 为基线 GMV（浏览量 V_i × 收缩转化率 θ_i × 均价 $\bar{p}_i$ ）， Δ_a 为动作 a 的预期提升系数， c_i 为置信度权重， R_i 为风险偏离度。风险惩罚项作用于增量 GMV 本身而非外部成本，体现了保守型决策偏好。

$$\text{s.t.} \quad \sum_{a \in A} x_{i,a} \leq 1, \quad \forall i \in C \quad (5.27)$$

$$\sum_{i \in C} \sum_{a \in A} x_{i,a} \cdot \text{Cost}(i, a) \leq B \quad (5.28)$$

$$\sum_{i \in C} \sum_{a \in A_{\text{slot}}} x_{i,a} \leq S \quad (5.29)$$

$$\sum_{i \in C_g} \sum_{a \in A} x_{i,a} \leq C_{\text{cap}}, \quad \forall g \in \mathcal{G}_{\text{category}} \quad (5.30)$$

$$\sum_{i \in C_b} \sum_{a \in A} x_{i,a} \leq B_{\text{cap}}, \quad \forall b \in \mathcal{B}_{\text{brand}} \quad (5.31)$$

其中, $x_{i,a} \in \{0,1\}$ 为是否对商品 i 选择动作 a 的二进制变量; B 为营销总预算; S 为展示插槽上限; C_{cap}, B_{cap} 分别为类目与品牌的决策次数天花板。

求解层首先寻找 Gurobi 求解器 (gurobipy) 进行全局整数规划迭代。若宿主机没有安装授权证书, 系统会自动降级触发基于边际 ROI 降序排列的贪心放置算法 (solve_with_greedy), 输出局部最优的分配方案并标记求解状态为 degraded_greedy。

5.3.2 多路召回与轻量排序推荐系统

推荐系统模块 (spark_jobs/recommendation.py) 是平台的核心业务输出模块之一, 其目标是在 Spark 分布式计算框架下, 为每个活跃用户会话生成一个高质量的 Top-K 商品推荐列表。与传统离线推荐系统不同, 本模块采用”近线 (nearline)”架构模式: 以当前清洗后的事件数据为输入, 在每次 Pipeline 运行时动态生成推荐结果, 并通过自动质量门禁与降级回滚机制保障推荐结果的可靠性。

模块设计遵循三个核心原则: 第一, 多路召回 (Multi-channel Recall) 确保候选商品来源的多样性与覆盖率; 第二, 轻量级排序器 (Lightweight Ranker) 在不引入外部模型服务的前提下, 通过规则评分与 Spark ML 逻辑回归模型的混合打分实现候选精排; 第三, 自动化质量门禁与提升/降级 (Promote/Degrade) 回滚机制确保不达标结果不会污染线上推荐快照。

5.3.2.1 个性化类目召回

该通道首先构建商品特征矩阵, 对每个商品计算加权转化评分:

$$S_{\text{product}} = 0.55 \times R_{v2p} + 0.25 \times R_{v2c} + 0.20 \times \min\left(\frac{RPV}{500}, 1\right) \quad (5.32)$$

其中, R_{v2p} 为浏览-购买转化率, R_{v2c} 为浏览-加购转化率, RPV 为单次浏览收入 (Revenue per View)。置信度则通过曝光量进行贝叶斯收缩:

$$C_{\text{product}} = \min\left(1.0, \sqrt{\frac{V}{500}}\right) \quad (5.33)$$

其中 V 为商品累计浏览量。该置信度因子使得低曝光商品的评分自然向均值收缩, 避免小样本高估。在每个一级类目内, 按 S_{product} 降序取 Top- N_{pool} 作为候选池 (默认 $N_{\text{pool}} = 80$)。

目标会话的提取则通过统计每个会话在各类目下的事件频次, 取事件最多且最新的类目作为该会话的偏好类目。最终, 候选商品按偏好类目与会话进行内连接 (inner join), 并通过 left_anti 连接排除用户已交互过的商品, 得到个性化候选集。商品特征矩阵构建代码如下:


```
product_features = (
    cleaned_df.groupby("product_id", "brand", "category_level1")
    .agg(
        views.alias("views"),
        carts.alias("carts"),
        purchases.alias("purchases"),
        revenue.alias("revenue"),
    )
    .withColumn("view_to_cart_rate", F.col("carts") / F.col("views"))
    .withColumn("view_to_purchase_rate", F.col("purchases") / F.col("views"))
    .withColumn("revenue_per_view", F.col("revenue") / F.col("views"))
    .withColumn("product_score", F.col("view_to_purchase_rate") * 0.55
        + F.col("view_to_cart_rate") * 0.25
        + F.least(F.col("revenue_per_view") / 500.0, F.lit(1.0)) * 0.20)
)
```

5.3.2.2 共现图谱邻居召回

该通道基于商品共现关联构建有向推荐图谱。对同一会话内出现的不同商品对 $(A \rightarrow B)$ 统计共现频次，并计算有向置信度与提升度：

$$\text{Confidence}(A \rightarrow B) = \frac{\text{Support}(A \cap B)}{\text{Sessions}(A)} \quad (5.34)$$

$$\text{Lift}(A \rightarrow B) = \frac{\text{Confidence}(A \rightarrow B)}{\text{Prior}(B)} = \frac{\text{Confidence}(A \rightarrow B)}{\text{Sessions}(B)/N_{\text{total}}} \quad (5.35)$$

其中 $\text{Prior}(B)$ 为目标商品 B 的全局先验概率。仅保留满足最小共现支持度（默认 ≥ 2 ）且提升度 ≥ 1.01 的边。对于目标会话中已浏览的商品，查找其图谱邻居作为推荐候选。候选综合评分融合了商品基础分、图谱置信度和亲和力分数：

$$S_{\text{graph}} = 0.75 \times S_{\text{product}} + 0.05 \times C_{\text{candidate}} + 0.25 \times A_{\text{affinity}} \quad (5.36)$$

$$A_{\text{affinity}} = \min \left(1.0, 0.65 \times \frac{\log(1 + \text{Lift})}{\log(11)} + 0.25 \times \text{Confidence} + 0.10 \times \min \left(\frac{\text{Support}}{20}, 1 \right) \right) \quad (5.37)$$

其中 $C_{\text{candidate}}$ 为融合了商品置信度与边置信度的复合置信度：

$$C_{\text{candidate}} = \min \left(1.0, 0.65 \times C_{\text{product}} + 0.35 \times \text{Confidence}(A \rightarrow B) \right) \quad (5.38)$$

5.3.2.3 全局优化兜底召回

当个性化召回与图谱召回的覆盖率不足时，兜底通道从全站热门商品池中选择 Top-40 商品作为保底候选。若上游优化模块（`optimization.py`）提供了促销计划，则对计划中的商品施加 +0.04 的分数加成，以确保运营策略在推荐结果中得到体现。

5.3.2.4 LR/GBT 混合排序器

三路候选合并后，系统首先通过基于规则的排序器进行初排，然后在训练数据充足时启用 Spark ML 排序模型进行精排。排序器的特征空间包含 7 维：

```
RANKER_FEATURE_COLUMNS = [
    "ranker_rule_score",      # 规则评分
    "ranker_confidence",     # 商品置信度
    "ranker_affinity_score",  # 图谱亲和力
    "ranker_fallback_flag",   # 是否兜底候选
    "ranker_graph_source_flag", # 是否图谱召回
    "ranker_personalized_source_flag", # 是否个性化召回
    "ranker_fallback_source_flag", # 是否优化兜底
]
```

训练标签基于历史交互：若用户会话对某商品存在购买行为，则 $y = 1$ ，否则 $y = 0$ 。模型默认采用 Logistic Regression（逻辑回归），亦可切换为 GBT（梯度提升树）。在训练行数不足（默认 < 50 ）或正负样本极端不平衡时，自动降级为纯规则排序器。

混合打分公式如下：

$$S_{\text{final}} = w \times P_{\text{ranker}} + (1 - w) \times \min(\max(S_{\text{rule}}, 0), 1) \quad (5.39)$$

其中 P_{ranker} 为排序模型输出的正类概率， S_{rule} 为规则评分， w 为混合权重（默认 $w = 0.7$ ）。最终每个会话按 S_{final} 降序取 Top- K （默认 $K = 5$ ）输出。排序窗口函数代码如下：

```
rank_window = Window.partitionBy("user_session").orderBy(F.desc("score"), F.desc("confidence"
↪ ))
recommendations = candidates.withColumn("rank", F.row_number().over(rank_window)).filter(F.
↪ col("rank") <= top_k)
```

5.3.2.5 ALS 隐式反馈基线与离线评估

为提供无偏的基线对照，模块实现了基于 Spark MLlib ALS（Alternating Least Squares）的隐式协同过滤推荐。交互权重按行为类型赋值：浏览 = 1、加购 = 3、购买 = 8。ALS 模型参数为 $\text{rank} = 8$ ， $\text{maxIter} = 5$ ， $\text{regParam} = 0.08$ ， $\alpha = 20.0$ 。

离线评估采用“留一法（Leave-One-Out）”切分策略：对每个会话，将其时间最近的一条交互作为测试集（holdout），其余作为训练集（train）。为防止信息泄漏，在生成评估候选时，显式地将 holdout 商品对从训练数据中移除后再执行召回。评估指标包括：

$$\text{Precision@}K = \frac{1}{|U|} \sum_{u \in U} \frac{|\hat{R}_u \cap R_u^*|}{K} \quad (5.40)$$

$$\text{Recall@}K = \frac{1}{|U|} \sum_{u \in U} \frac{|\hat{R}_u \cap R_u^*|}{|R_u^*|} \quad (5.41)$$

$$\text{NDCG@}K = \frac{1}{|U|} \sum_{u \in U} \frac{\text{DCG}_u}{\text{IDCG}_u}, \quad \text{DCG}_u = \sum_{i=1}^K \frac{\text{hit}_i}{\log_2(i+1)} \quad (5.42)$$

其中 $\hat{R}_u$ 为推荐列表, R_u^* 为 holdout 真实集合, IDCG_u 为理想排序下的折扣累计增益。

5.3.2.6 质量门禁与提升/降级回滚

每次推荐生成后, 系统执行 6 项质量门禁检查:

```
checks = [
    ("coverage_rate", coverage_rate, ">=", 0.75), # 会话覆盖率
    ("fallback_rate", fallback_rate, "<=", 0.80), # 兜底占比上限
    ("avg_confidence", avg_confidence, ">=", 0.03), # 平均置信度下限
    ("freshness_lag", freshness_lag, "<=", 5300000), # 新鲜度延迟
    ("duplicate_rate", duplicate_rate, "<=", 0.0), # 重复推荐率
    ("invalid_prod_rate", invalid_rate, "<=", 0.0), # 无效商品率
]
```

当所有门禁通过时, 新推荐快照被提升 (Promote) 为活跃快照, 旧快照自动归档为回滚快照。当门禁失败时, 系统自动执行降级 (Degrade) 操作: 加载上一次已通过的快照作为当前推荐结果, 并在 manifest 中记录 "degraded_previous_snapshot" 状态。该机制确保推荐服务不会因单次异常计算而输出低质量结果。

推荐系统的输入直接依赖数据清洗模块输出的 cleaned_df, 以及运营优化模块 (optimization.py) 输出的促销计划。

推荐结果被增长实验模块 (experimentation.py) 消费, 用于评估个性化推荐策略在对照实验中的提升效果。

推荐覆盖指标 (覆盖率、兜底率、平均置信度) 被实验护栏模块实时监控。

5.4 商品组合智能与用户旅程分析

5.4.1 HHI 集中度与品类结构健康度

商品组合模块 (spark_jobs/portfolio.py) 从品类结构、品牌混合、价格带分布和单品集中度四个维度评估平台商品组合的健康度。其核心目标是识别结构

风险（如过度依赖单一品类或头部单品）和增长机会（如高流量低转化的价格带缺口），为商品运营策略提供数据驱动的决策依据。

5.4.1.1 赫芬达尔-赫希曼指数

系统采用赫芬达尔-赫希曼指数（Herfindahl-Hirschman Index, HHI）衡量收入在单品层面的集中度。对商品集合 $\{p_1, p_2, \dots, p_n\}$ ，每个商品的收入份额为 s_i ，HHI 计算公式为：

$$HHI = \sum_{i=1}^n s_i^2 = \sum_{i=1}^n \left(\frac{\text{Revenue}_i}{\text{Revenue}_{\text{total}}} \right)^2 \quad (5.43)$$

HHI 的取值范围为 (0, 1]：当收入完全集中在单一商品时 $HHI = 1$ ；当收入均匀分布在 n 个商品时 $HHI = 1/n$ 。HHI 越高表示品类结构越脆弱，头部单品波动可能导致整体收入剧烈起伏。在 Spark 实现中，先计算每个商品的收入份额平方，再通过聚合函数求和：

```
scored = (
  product.crossJoin(totals)
  .withColumn("revenue_share",
    F.col("revenue") / F.col("_total_revenue"))
  .withColumn("hhi_contribution",
    F.round(F.col("revenue_share") * F.col("revenue_share"), 8))
)
```

5.4.1.2 价格带分析

系统将购买事件按价格区间划分为四个标准价格带：

$$\text{PriceBand}(p) = \begin{cases} \text{budget} & p < 50 \\ \text{mass} & 50 \leq p < 200 \\ \text{premium} & 200 \leq p < 1000 \\ \text{luxury} & p \geq 1000 \end{cases} \quad (5.44)$$

在每个品类 $\times$ 价格带的交叉维度上，系统统计购买量、收入、均价和份额占比。价格带分布的集中程度反映了品类的定价策略多样性：若某品类 $> 35\%$ 的收入集中在单一价格带，则标记为 **concentration_risk**（集中风险）。

5.4.1.3 品类混合与机会识别

品类混合分析以一级品类为粒度，聚合浏览、加购、购买和收入指标，并计算各品类的收入份额和购买份额。系统在品类维度上自动识别三类运营机会：

1. **流量转化缺口 (traffic_conversion_gap)**: 浏览量 ≥ 100 但浏览-购买转化率 $< 2\%$ 的品类, 表明存在流量浪费, 需优化详情页或推荐策略。机会影响力评分为:

$$I_{\text{gap}} = \text{Revenue} \times \max(0.02 - R_{v2p}, 0.001) \quad (5.45)$$

2. **品类集中风险 (concentration_risk)**: 收入份额 $\geq 35\%$ 的品类, 表明平台对该品类过度依赖。
3. **价格带结构机会 (price_band_mix)**: 基于品类 $\times$ 价格带交叉维度的收入池分布, 识别未充分开发的价格区间。

5.4.1.4 质量门禁

模块内置四项质量检查: 最小购买行数 (≥ 100)、最小历史跨度 (≥ 7 天)、有效价格购买率 ($\geq 98\%$) 和价格带覆盖数 (≥ 2)。任一检查未通过则标记为 `needs_review`, 提醒分析结果可能存在偏差。

商品组合模块的输入为清洗后的事件流 (`cleaned_df`)。输出的品类结构分析和价格带缺口为运营优化模块 (`optimization.py`) 的候选商品选择提供结构化依据。HHI 集中度指标可与异常监控模块联动, 当头部商品出现异常波动时自动评估其对整体收入的影响范围。

5.4.2 用户旅程与转移矩阵

用户旅程模块 (`spark_jobs/journey.py`) 从原始事件流中还原用户在单一会话内的完整行为路径, 并通过转移概率分析与退出事件统计为体验优化提供数据支撑。与前述“高频行为路径提取”侧重路径签名统计不同, 本模块进一步构建了转移事实表 (Transition Facts), 将相邻行为转换为有向边并统计转化率, 从而识别用户流失的关键节点。

5.4.2.1 会话路径构建

系统对每个 `user_session` 聚合其所有事件, 按时间戳升序排序生成行为序列。核心实现使用 Spark 的 `collect_list` 与 `sort_array` 组合算子:

```
event_struct = F.struct(
    F.col("event_timestamp").alias("ts"),
    F.col("event_type").alias("event_type"),
    F.col("product_id").cast("string").alias("product_id"),
    F.col("category_level1").alias("category_level1"),
)
base = (
    cleaned_df.filter(F.col("user_session") != "unknown")
    .groupBy("user_session")
```

```
.agg(
  F.sort_array(F.collect_list(event_struct)).alias("event_structs"),
  F.max(F.when(F.col("event_type") == "purchase", F.lit(1))
        .otherwise(F.lit(0))).alias("has_purchase"),
)
.withColumn("events", F.expr(
  "transform(event_structs, x -> x.event_type)"
))
.withColumn("path_signature",
  F.concat_ws(" -> ", F.expr("slice(events, 1, 8)")))
)
```

每个会话同时提取以下衍生特征：路径签名（前 8 个事件的类型序列）、首事件与末事件类型、总步数、会话时长（秒）、是否包含购买和加购行为。

5.4.2.2 转移事实矩阵与转化率

转移事实表通过将每个会话的事件序列按相邻位置拆分为 (e_i, e_{i+1}) 有序对来构建。使用 Spark 的 `arrays_zip` 函数实现高效分布式拆分：

$$T(e_i, e_{i+1}) = \sum_{\text{sessions}} \mathbb{1}[\text{session contains transition } e_i \rightarrow e_{i+1}] \quad (5.46)$$

对每条转移边，系统额外统计会话数、含购买会话数和关联收入，计算转移级转化率：

$$R_{\text{conv}}(e_i \rightarrow e_{i+1}) = \frac{\text{PurchaseSessions}(e_i \rightarrow e_{i+1})}{\text{Sessions}(e_i \rightarrow e_{i+1})} \quad (5.47)$$

当目标事件为 `remove_from_cart` 或转化率低于 2% 时，系统自动标记该转移为 `"inspect friction"`（摩擦点排查），提示运营团队关注。

5.4.2.3 退出事件分析

退出事件分析以每个会话的最后行为（`last_event`）为分组维度，统计各退出类型的会话占比、购买率和关联收入：

$$\text{ExitShare}(e) = \frac{\text{Sessions ending with } e}{\text{Total Sessions}} \quad (5.48)$$

高占比但低购买率的退出类型（如 `view` 退出）是用户流失的关键信号，应优先进行体验优化。

旅程模块的输入为清洗后的事件流（`cleaned_df`），输出的转移事实表和路径统计被前端桑基图和漏斗图渲染。退出事件分析结果可与异常模块联动，当特定退出类型的占比出现异常波动时触发告警。购买路径统计为推荐系统的候选质量评估提供间接验证依据。

5.5 增长实验平台与因果推断框架

增长实验模块（`spark_jobs/experimentation.py`）是平台的决策科学基础设施，旨在为推荐策略、生命周期运营策略和商品运营位策略提供可控的 A/B 实验分组与效果评估框架。与生产级在线实验平台不同，本模块在课程实验环境下采用“离线历史回放（Offline History Replay）”模式：基于已有的用户行为数据，通过哈希分桶确定性分组模拟随机化实验，并使用历史指标作为实验结果的代理变量。尽管如此，模块在统计方法论上严格遵循在线实验的工程规范，包括样本比例失配检验（SRM）、双比例 z 检验、置信区间估计和分位提升分析（AUUC）。

5.5.1 确定性哈希分桶与样本比例检验

5.5.1.1 哈希分桶与实验分组

系统对每个实验定义了一个独立的分桶空间。用户的实验组分配由用户 ID 与实验键的联合哈希值决定：

$$B(u, e) = \frac{|\text{hash}(\text{user_id}, \text{experiment_key})| \bmod 10000}{10000} \quad (5.49)$$

其中 $B(u, e) \in [0, 1)$ 为均匀分布的分桶值。分配规则为：

$$\text{Variant}(u, e) = \begin{cases} \text{treatment} & \text{if } B(u, e) < \tau \\ \text{control} & \text{otherwise} \end{cases} \quad (5.50)$$

其中 τ 为实验流量分割比（默认 $\tau = 0.5$ ）。该哈希分配方式具有确定性（同一用户在同一次运行中始终分配到同一组）和跨实验独立性（不同实验键的哈希空间互不干扰）。

5.5.1.2 样本比例失配检验（SRM）

在评估实验结果之前，系统首先检验分组是否符合预期比例。SRM 检验使用卡方统计量：

$$\chi_{\text{SRM}}^2 = \frac{(N_T - E_T)^2}{E_T} + \frac{(N_C - E_C)^2}{E_C} \quad (5.51)$$

其中 N_T 和 N_C 分别为实验组和对照组的实际用户数， $E_T = N \cdot \tau$ 和 $E_C = N \cdot (1 - \tau)$ 为期望值。 p 值通过互补误差函数计算：

$$p_{\text{SRM}} = \text{erfc} \left(\sqrt{\frac{\chi_{\text{SRM}}^2}{2}} \right) \quad (5.52)$$

当 $p_{\text{SRM}} < \alpha_{\text{SRM}}$ (默认 $\alpha_{\text{SRM}} = 0.001$) 时, SRM 检验失败, 表明分组存在严重失衡, 实验结果不可信。对应代码实现如下:

```
expected_treatment = total * treatment_split
expected_control = total * (1.0 - treatment_split)
chi_square = 0.0
if expected_treatment:
    chi_square += ((treatment_users - expected_treatment) ** 2) / expected_treatment
if expected_control:
    chi_square += ((control_users - expected_control) ** 2) / expected_control
srm_p_value = math.erfc(math.sqrt(chi_square / 2))
```

5.5.1.3 假设检验与分位提升分析

5.5.1.4 双比例假设检验与提升估计

在 SRM 通过后, 系统对每个实验的主要指标 (如购买率) 执行双比例 z 检验。绝对提升定义为:

$$\Delta = \hat{p}_T - \hat{p}_C \quad (5.53)$$

标准误由两独立比例的方差之和计算:

$$SE = \sqrt{\frac{\hat{p}_T(1 - \hat{p}_T)}{N_T} + \frac{\hat{p}_C(1 - \hat{p}_C)}{N_C}} \quad (5.54)$$

z 统计量对应的双尾 p 值为:

$$p = \text{erfc}\left(\frac{|\Delta|}{SE \cdot \sqrt{2}}\right) \quad (5.55)$$

95% 置信区间为:

$$CI_{95\%} = [\Delta - 1.96 \cdot SE, \Delta + 1.96 \cdot SE] \quad (5.56)$$

实验决策规则按优先级依次为: 样本不足则返回 `needs_more_sample`; SRM 失败则返回 `blocked_by_srm`; 标准误非正值导致 p 值不可计算时返回 `not_measurable`; $p \leq 0.05$ 且 $\Delta > 0$ 则返回 `positive_significant`; $p \leq 0.05$ 且 $\Delta < 0$ 则返回 `negative_significant`; 否则返回 `not_significant`。

5.5.1.5 分位提升分析与 AUUC 估计

为评估实验在不同用户分群上的异质性效果, 系统将用户按 `uplift_score` 降序分为 10 个分位桶 (decile), 分别计算每个桶内实验组与对照组的转化率之差:

$$\text{Uplift}_d = \hat{p}_{T,d} - \hat{p}_{C,d} \quad (5.57)$$

累计增益为：

$$\text{Gain}_d = \sum_{j=1}^d \text{Uplift}_j \cdot N_{T,j} \quad (5.58)$$

AUUC (Area Under Uplift Curve) 作为提升曲线下的面积，通过对所有分位的累计增益求和近似：

$$\text{AUUC} = \sum_{d=1}^{10} \text{Gain}_d \quad (5.59)$$

AUUC 值越高，说明实验对高潜力用户的提升效果越显著。需注意的是，由于本模块使用离线历史回放而非随机化暴露，AUUC 的因果解释性受限，因此系统在输出中明确标注 `causal_valid: False`。

5.5.1.6 实验护栏机制与上下游关系

5.5.1.7 五项护栏检查

系统内置五项护栏检查以确保实验质量：最小分组用户数（默认 ≥ 50 ）、最小实验组用户数（ ≥ 20 ）、最小对照组用户数（ ≥ 20 ）、推荐平均置信度（ ≥ 0.01 ），以及分段变体不平衡度（ ≤ 0.2 ）。不平衡度按生命周期分段计算：

$$\text{Imbalance}_s = \frac{|N_{T,s} - N_{C,s}|}{N_{T,s} + N_{C,s}} \quad (5.60)$$

任一护栏检查失败，整体护栏状态标记为 `needs_review`。

5.5.1.8 与上下游模块的关系

实验平台的输入依赖客户生命周期模块 (`lifecycle.py`) 输出的用户分群标签 (生命周期段与风险带)，以及推荐系统模块 (`recommendation.py`) 输出的推荐覆盖指标。实验结果 (包括分组分配、提升估计和 AUUC) 被前端面板渲染为实验报告，为运营决策提供统计学依据。

5.6 自动决策引擎与受控查询

自动决策引擎 (`spark_jobs/decision_maker.py`) 是平台的顶层治理模块，其职责是将异常监控模块 (`anomaly.py`) 和运营优化模块 (`optimization.py`) 的输出转化为可执行的自动干预决策与补货工单。该模块的核心设计理念是”异常驱动干预、预测驱动补货”：当异常雷达检测到严重的销量或收入崩塌时，自动触发防御性拦截；当预测模块发现未来需求将超出当前库存水位时，自动发起预测性补

货建议。

5.6.1 异常驱动的自动干预决策

5.6.1.1 三级干预分流规则

干预决策逻辑按异常告警的严重程度 (**severity**)、方向 (**direction**) 和指标类型 (**metric**) 进行分级分流。仅当告警级别为 **critical** 且方向为 **drop** (暴跌) 时触发自动干预。决策分流规则如下：

1. **商品级销量/收入暴跌** → **emergency_suspend** (紧急挂起)。当异常实体类型为 **product** 且指标为 **revenue** 或 **purchases** 时，触发商品下架或暂挂决策，防止因标错价、系统缺陷或缺货导致的持续性损失。证据强度由稳健 z 分数 z_{robust} 量化。
2. **品类级收入崩塌** → **price_audit** (价格审计)。当异常实体类型为 **category** 时，触发整条品类的价格配置与支付漏斗安全排查，排除推荐引擎失效或全局支付故障。
3. **转化率异常突降** → **funnel_check** (漏斗诊断)。当指标为 **conversion_rate** 或 **view_to_purchase_rate** 时，触发详情页加载与结算流程的灰度验证。

5.6.1.2 决策证据量化

每条干预决策附带结构化的证据载荷 (**evidence payload**)，包含触发异常的实体 ID、稳健 z 分数绝对值、观测期间的指标均值与标准差，以及建议的排查优先级。决策清单写入运行 **Manifest**，供前端决策面板展示和后续审计。

5.6.1.3 预测驱动的智能补货

5.6.1.4 需求预测与库存水位

补货决策综合考虑运营优化模块的促销计划和预测模块的品类增长系数。对每个促销候选商品，系统计算未来 7 天的预期总需求量：

$$D_{\text{forecast}} = D_{\text{baseline}} + \Delta D_{\text{incremental}} \quad (5.61)$$

其中 D_{baseline} 为无促销条件下的基准预测销量（由基准 GMV 除以均价得到）， $\Delta D_{\text{incremental}}$ 为促销活动带来的增量销量。当前库存水位以历史购买量的 1.0 倍模拟。

5.6.1.5 补货量与成本估算

当预期需求超出当前库存时，触发补货决策，补货量计算公式为：

$$Q_{\text{restock}} = \max(1, \lceil D_{\text{forecast}} \times 1.5 - S_{\text{current}} \rceil) \quad (5.62)$$

其中 S_{current} 为当前周转库存，1.5 为安全库存系数。补货成本估算为：

$$C_{\text{restock}} = Q_{\text{restock}} \times P_{\text{avg}} \times 0.6 \quad (5.63)$$

其中 P_{avg} 为商品均价，0.6 为进货成本占售价的比例系数。系统根据品类特征分配不同的到货周期 (lead time) 与供应商：电子品类 4 天、快时尚品类 2 天、美妆品类 3 天。当检测到销量突增信号 (is_spike) 时，安全库存系数自动上调至 2.5 以防止断货。此外，突增场景下到货周期自动缩短 1 天 ($\text{lead_time} = \max(1, \text{lead_time} - 1)$)，实现紧急加急补货。补货工单按成本从高到低排序，帮助采购团队确定轻重缓急。

决策引擎不直接执行 Spark 计算，而是作为纯逻辑聚合层，消费异常模块的 critical 告警和优化模块的促销计划。输出的决策清单 (intervention_decisions 和 restock_decisions) 被写入运行 Manifest，供前端决策面板展示和后续审计。

5.6.1.6 受控自然语言查询

受控自然语言查询模块允许运营人员以结构化的自然语言模板（如”查询某品类近 7 天转化率”）向系统发起查询，系统将其翻译为受控的 Spark SQL 语句执行。查询模板经过参数白名单校验与 SQL 关键字过滤，确保查询安全可控，避免任意 SQL 注入风险。该模块降低了非技术运营人员使用数据平台的门槛，同时维护了数据访问的治理边界。

第 6 章 运维风险控制与 Benchmark 评估

6.1 异常检测与实时预警

6.1.1 Robust Z-Score 异常检测

为防范促销恶意刷量或流量瞬时爆降等行为对分析大盘产生的严重偏离误导，系统设计了稳健化（Robust）异常监控逻辑。传统的均值-标准差方法易受大幅离群值的影响，从而把阈值反向拉宽导致漏报。系统引入了绝对中位数偏差（MAD, Median Absolute Deviation）统计量：

$$\text{MAD}(X) = \text{median}(|x_i - \text{median}(X)|) \quad (6.1)$$

$$z_{\text{robust}} = \frac{|x_i - \text{median}(X)|}{1.4826 \cdot \text{MAD}(X)} \quad (6.2)$$

其中，常数因子 1.4826 使 MAD 在正态分布下对齐标准差。当 $z_{\text{robust}} > 6.0$ 时判定为 **critical** 严重异常，当 $z_{\text{robust}} \in (3.5, 6.0]$ 判定为 **warning**，当 $z_{\text{robust}} \in (2.5, 3.5]$ 判定为 **watch**。实际判定还需满足显著性门槛：例如收入指标需 $R \geq 500$ 才触发 **critical**， $R \geq 150$ 才触发 **warning**；转化率指标需浏览量 ≥ 30 且购买量 ≥ 3 。

6.1.1.1 多机制融合的异常判定策略

系统在基础 Robust Z-Score 之上融合了五项增强机制以降低误报率。第一，**工作日季节性中位数基线**：按星期几分区计算历史同期中位数，消除周末/工作日周期性波动对阈值的干扰。第二，**预测增强时序对齐**：当需求预测模块输出有效预测值时，以预测值替代历史中位数作为期望基线，并使用预测置信区间判定越界，使异常判定具备前瞻性。第三，**全站大盘动态放大因子（site_factor）**：在大促期间全站指标整体抬升时，动态放大阈值以消除假阳性。第四，**MAD 分指标动态下限锚位**：针对不同指标类型设置差异化 MAD 下限（收入 ≥ 100 、浏览量 ≥ 20 、购买量 ≥ 2 、比率 ≥ 0.01 ），防止稀疏商品的 MAD 过小导致 z-score 爆炸。第五，**归零检测**：当历史基线超过最小观变量（ $\text{baseline} \geq \text{min_volume}$ ）而当日值为零时，直接触发告警，无须经过 z-score 计算。

6.2 生命周期分层与实验分流

在运维风险控制中，用户的生命周期分层与实验分流起到了核心的流量隔离和质量监控作用。具体的分流算法与统计学假设检验（包括确定性哈希分桶、样本比例失配检验、双比例假设检验及分位提升分析等）已在第 5 章中详细阐述。本节重点从系统运行稳定性、自动风控和实验护栏的角度讨论其在运维中的风险防控与干预机制。

6.2.1 生命周期分层与运维干预

生命周期管理基于用户单日特征表，统计活跃天数、消费频次和最近访问距今日期（Recency），应用确定性规则链将用户划分为 **Champion**（冠军用户）、**Loyal**（忠诚用户）、**Browser**（纯浏览者）等类别，并基于最近活跃天数和加购记录计算风险带（**Risk Band**）。这些生命周期段与风险带标签不仅是业务上的分组依据，也是运维自动决策引擎（第 5.5 节）触发差异化自动干预决策与预测性补货工单的核心控制变量。

6.2.2 实验分流的运维稳定机制与护栏监控

对于离线历史回放实验，如何在分布式环境中保持分流的幂等性以及如何控制统计学评估中的假阳性风险，是系统运维保障的核心挑战：

- 1. 流量单调稳定性控制：**用户的实验组分配由用户 ID 与实验键的联合哈希分桶值（参见公式（5.49））决定。该分配方法具有完全确定性，即同一用户在多次计算管道运行中始终被分配到同一组，从物理上避免了用户在页面交互中因分组漂移导致体验错乱，并且在 **Executor** 侧以纯哈希算子无状态并行运算，消除了集中式分流服务的单点瓶颈。
- 2. SRM 卡方检验防护：**系统在离线回放模块运行结束后，会自动执行样本比例失配检验（**SRM**）（参见公式（5.51））。若检验 p 值小于设定门限（默认 0.001），表明实验流量分桶发生偏斜（如因部分数据丢失或哈希碰撞引发比例偏离），系统将在前端 **Ops** 页面突出展示 **SRM** 拦截警报，并强行阻断本次实验结果快照的提拔，保障分析证据链的可靠性。
- 3. 实验护栏与不平衡监控：**为防范小样本或稀疏特征条件下的统计估计失真，系统内置了五项护栏检查（包含最小样本数、置信度下限及基于公式（5.60）的分段变体不平衡度）。一旦护栏状态标记为 `needs_review`，或在离线回放中由于非随机暴露导致因果有效性不可靠时（输出 `causal_valid: False`），系统会自动向自动决策引擎发出警告，并拒绝将新计算 of 推荐参数提升为

线上活跃状态，由运维人员介入审计。

6.3 可追溯 Ops 证据链

6.3.1 Ops 证据看板与运行元数据

可追溯（Ops）证据看板提取运行产生的指标元数据。它不重新对数据行执行分布式运算，而是流式拼接运行 manifest 中的输入快照 MD5、HDFS Parquet 写入信息与 YARN 任务记录，为答辩演示提供科学证据。

6.3.1.1 旧数据治理策略

在旧数据治理上，项目实行”保留输入快照及最后 event log，定时清洗中间产物”的精简策略，规定保留 1%/5% 样本 CSV 文件及最终 benchmark 的 6 个 event log，早期重复烟雾测试日志由清理脚本强制物理清除，以此保障容器运行环境的空间健康水位。存储快照如图6.1所示。

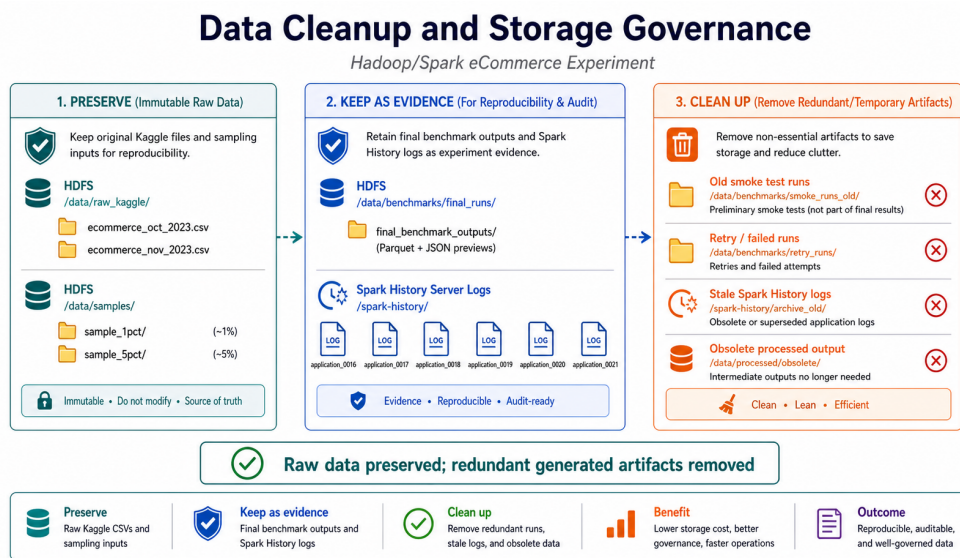


图 6.1 旧数据清理后的存储治理示意图

6.4 多维度 Benchmark 实验结果与对比分析

6.4.1 多配置对照实验结果

系统在 1% 样本下执行了五组对照实验，并在 5% 数据集下进行了压力扩展性验证。所有 YARN 提报作业最终均返回 SUCCEEDED。实验结果汇总如表6.1所示。

如图6.2与图6.3所示，比对 YARN-only (964.889s) 和 YARN+AQE (538.657s)，

表 6.1 1% 与 5% benchmark 实验结果

样本	对照组	耗时 (s)	输入行	输出行	Application ID
1%	Baseline local CSV	198.843	1,104,709	1,103,488	local-1781015058333
1%	YARN-only CSV	964.889	1,104,709	1,103,488	application_1780991452919_0016
1%	YARN + AQE CSV	538.657	1,104,709	1,103,488	application_1780991452919_0017
1%	YARN + algorithm CSV	535.804	1,104,709	1,103,488	application_1780991452919_0018
1%	YARN + Parquet	573.183	1,103,488	1,103,488	application_1780991452919_0019
5%	YARN + algorithm CSV	1875.099	5,412,207	5,405,748	application_1780991452919_0020
5%	YARN + Parquet	1828.329	5,405,748	5,405,748	application_1780991452919_0021

耗时缩短近 44%，验证了 AQE 调优的关键作用。

上述结果有力地支持了以下论点：如果不配置执行计划优化，仅加入分布式 YARN 进行资源调度，反而会由于任务拉起的开销而降低系统整体吞吐率。

前端 Ops 证据面板集成



图 6.2 前端 Ops 证据面板集成示意图

YARN + Spark 实验对照矩阵

	1 Local CSV Baseline	2 YARN-only CSV	3 YARN + AQE	4 YARN + Algorithm Guards	5 YARN + Parquet	结论
输入数据	CSV 文件 (本地文件系统)	CSV 文件 (分布式存储)	CSV 文件 (分布式存储)	CSV 文件 (分布式存储)	Parquet 文件 (分布式存储)	YARN 只提供 资源调度；
调度方式	本地模式 (单机进程内)	YARN 调度 (资源分配与管理)	YARN 调度 (资源分配与管理)	YARN 调度 (资源分配与管理)	YARN 调度 (资源分配与管理)	AQE 与算法限流 降低 OOM 风险；
优化开关	AQE 关闭 算法限流 关闭	AQE 关闭 算法限流 关闭	AQE 开启 算法限流 关闭	AQE 开启 算法限流 开启	AQE 开启 算法限流 开启	Parquet 改善 列式读取与 落盘布局。
输出证据	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	执行日志 物理计划 (EXPLAIN) 错误与告警日志	

图 6.3 YARN + Spark 实验对照矩阵

6.4.1.1 基于 HDFS rolling event log 的 Spark 运行指标补采集

由于 Spark History Server 内置的 API 在读取部分极其庞大的复杂 Stage 记录时偶发异常，项目开发了 rolling event log 补采集脚本。该脚本绕过 History API，直接在 HDFS 中解压并流式扫描 .zstd event log 日志，从而补采集了真实的 Shuffle 读写与 Spill 数据，汇总如表6.2所示。

表 6.2 Spark event log 指标补采集结果

Application ID	Tasks	Failed	Retried	Shuffle Read	Memory Spill
application_1780991452919_0016	6,290	0	0	9.4GB	2.1GB
application_1780991452919_0017	4,526	0	0	9.4GB	728.0MB
application_1780991452919_0018	4,526	0	0	9.4GB	728.0MB
application_1780991452919_0019	5,057	0	0	9.4GB	792.5MB
application_1780991452919_0020	5,149	0	0	44.9GB	89.6GB
application_1780991452919_0021	5,815	0	0	44.9GB	89.4GB

6.4.1.2 典型模块计算效能 Benchmark 评估与结论

为验证特定算法的局部耗时表现，系统使用 1% 样本中的 200,000 行典型数据，对关联分析、推荐召回、异常监控和实验分流四个计算密集型 Pipeline 进行了局部效能压测，结果如表6.3所示。

表 6.3 典型部分数据模块 benchmark 结果

模块	任务	输入行	输出行	耗时 (s)
Affinity	affinity_pipeline	199,895	80	3.852
Recommendation	recommendation_pipeline	199,895	1,000	1.740
Anomaly	anomaly_pipeline	199,895	644	7.865
Experimentation	experimentation_pipeline	199,895	21,716	8.703

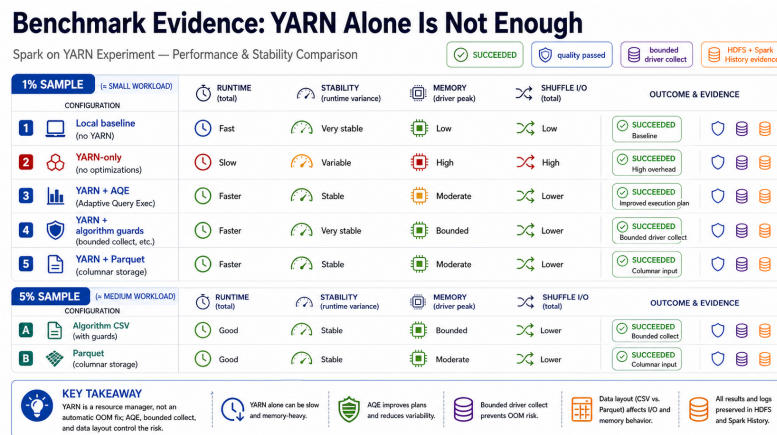


图 6.4 Benchmark 证据概览（精确数值见表6.1）

如图6.4所示，综上所述，本实验对照研究得出以下核心结论：第一，将分布式提报作业交付 YARN 不会自动化解性能与物理溢出瓶颈，甚至会引入非必要的网络开销，因此执行层调优与算法上限控制必不可少；第二，AQE 对倾斜连接的动态优化可显著缩短计算长尾；第三，Parquet 列式布局可在更大数据样本下显著加快读取。本系统已在本机资源约束内最大化消除了 OOM 崩溃，完全达到了设计预期。

第 7 章 致谢

本课程设计能够完成，首先感谢任课老师在大数据与云计算课程中对 Hadoop、Spark、分布式存储和集群实验方法的讲解，使本项目能够从普通本地数据分析扩展到 Hadoop YARN 实验环境。老师在课程中强调实验证据、运行截图和结果解释的重要性，也促使本项目不只停留在功能演示，而是进一步补充 benchmark、质量门禁和 Ops 展示。

感谢同学们在调试 Docker、HDFS、YARN、Spark submit、前端页面和实验报告过程中提供的交流与帮助。尤其是在 ARM Mac 架构、Docker 镜像选择、端口冲突、Spark History 日志异常和前端展示问题上，同学之间的讨论帮助我更快定位了问题边界。

同时感谢开源社区提供的 Hadoop、Spark、Docker、Flask、React、Vite、TanStack Query、Matplotlib 等工具。没有这些成熟工具，本课程设计很难在本机环境中完成从原始数据、分布式计算、后端 API 到前端运维展示的完整闭环。

参考文献

- [1] Dean J, Ghemawat S. Mapreduce: Simplified data processing on large clusters[C]//Proceedings of the 6th Symposium on Operating Systems Design and Implementation. 2004: 137-150.
- [2] Vavilapalli V K, Murthy A C, Douglas C, et al. Apache hadoop yarn: Yet another resource negotiator[C]//Proceedings of the 4th Annual Symposium on Cloud Computing. 2013: 1-16.
- [3] Zaharia M, Chowdhury M, Das T, et al. Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing[C]//Proceedings of the 9th USENIX Symposium on Networked Systems Design and Implementation. 2012: 15-28.
- [4] Zaharia M, Xin R S, Wendell P, et al. Apache spark: A unified engine for big data processing [J]. Communications of the ACM, 2016, 59(11): 56-65.
- [5] Armbrust M, Xin R S, Lian C, et al. Spark sql: Relational data processing in spark[C]//Proceedings of the 2015 ACM SIGMOD. 2015: 1383-1394.
- [6] Herodotou H, Lim H, Luo G, et al. Starfish: A self-tuning system for big data analytics[C]//Proceedings of the 5th Biennial Conference on Innovative Data Systems Research. 2011: 261-272.
- [7] Kwon Y, Balazinska M, Howe B, et al. Skewtune: Mitigating skew in mapreduce applications[C]//Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data. 2012: 25-36.
- [8] Rasmussen A, Porter G, Conley M, et al. Themis: An i/o-efficient mapreduce[C]//Proceedings of the Third ACM Symposium on Cloud Computing. 2012: 13:1-13:14.
- [9] Docker Inc. Docker compose documentation[EB/OL]. 2026. <https://docs.docker.com/compose/>.